1 Program Analysis

Goal of This Chapter Before we dive into the way static analysis tools operate, we need to define their scope and describe the kinds of questions they can help solve. In section 1.1 we discuss the importance of understanding the behavior of programs by semantic reasoning, and we show applications in section 1.2. Section 1.3 sets up the main concepts of static analysis and shows the intrinsic limitations of automatic program reasoning techniques based on semantics. Section 1.4 classifies the main approaches to semantic-based program reasoning and clarifies the position of static analysis in this landscape.

Recommended Reading: **[S]**, **[D]**, **[U]**

1.1 Understanding Software Behavior

In every engineering discipline, a fundamental question is, will our design work in reality as we intended? We ask and answer that question when we design mechanical machines, electrical circuits, or chemical processes. The answer comes from analyzing our designs using our knowledge about nature that will carry out the designs. For example, using Newtonian mechanics, Maxwell equations, Navier-Stokes equations, or thermodynamic equations, we analyze our design to predict its actual behavior. When we design a bridge, for example, we analyze how nature runs the design, namely, how various forces (e.g., gravitation, wind, and vibration) are applied to the bridge and whether the bridge structure is strong enough to withstand them.

The same question applies to computer software. We want to ensure that our software will work as intended. The intention for analysis ranges widely. For general reliability, we want to ensure that the software will not crash with abrupt termination. If the software interacts with the outside world, we want to ensure it will not be deceived to violate the host computer's security. For specific functionalities, we want to check if the

software will realize its functional goal. If the software is to control cars, we want to ensure it will not drive them to an accident. If the software is to learn our preference, we want to ensure it will not degrade as we teach more. If the software transforms the medical images of our bodies, we want to ensure it will not introduce false pixels. If the software is to bookkeep the ledgers for crypto currency, we want to ensure it will not allow double spending. If the software translates program text, we want to ensure the source's meaning is not lost in translation.

There is, however, one difference between analyzing software and analyzing other types of engineering designs: for computer software, it is not nature that will run the software but the computer itself. The computer will execute the software according to the meanings of the software's source language. Software's run-time behavior is solely defined by the meanings of the software's source language. The computer is just an undiscerning tool that blindly executes the software exactly as it is written. Any execution behavior that deviates from our intention is because the software is mistakenly written to behave that way.

Hence, for computer software, to answer the question of whether our design will work as we intended, we need knowledge by which we can somehow analyze the meanings of software source language. Such knowledge corresponds to knowledge that natural sciences have accumulated about nature. We need knowledge that computer science has accumulated about handling the meanings of software source languages.

We call a formal definition of a software's run-time behavior, which is determined by its source language's meanings, *semantics*:

Definition 1.1 **(Semantics and semantic properties)** *The* semantics *of a program is a (generally formal—although we do not make it so in this chapter) description of its run-time behaviors. We call* semantic property *any property about the run-time behavior (semantics) of a program.*

Hence, *checking if a software will run as we intended* is equivalent to *checking if this software satisfies a semantic property of interest.*

In the following, we call a technique to check that a program satisfies a semantic property *program analysis*, and we refer to an implementation of program analysis as a *program analysis tool*.

Figure 1.1 illustrates the correspondence between program analysis and design analysis of other engineering disciplines.

1.2 Program Analysis Applications and Challenges

Program analysis can be applied wherever understanding program semantics is important or beneficial. First, software developers (both humans and machines) may be the biggest beneficiaries. Software developers can use program analysis for quality assurance, to locate errors of any kind in their software. Software maintainers can use program analysis to understand legacy software that they maintain. System security gatekeepers can use program analysis to proactively screen out programs whose semantics can be malicious.

Software that handles programs as data can use program analysis for the programs' performance improvement too. Language processors such as translators or compilers need program analysis to translate the input programs into optimized ones. Interpreters, virtual machines, and query processors need program analysis for optimized execution of the input programs. Automatic program synthesizers can use program analysis to check and tune what they synthesize. Mobile operating systems need to understand an application's semantics in order to minimize the energy consumption of the application. Automatic tutoring systems for teaching programming can use program analysis to hint to students a direction to amend their faulty programs.

	Computing area	Other engineering areas
Object	Software	Machine/building/circuit/chemical process design
Execution subject	Computer runs it	Nature runs it
Our question	Will it work as intended?	Will it work as intended?
Our knowledge	Program analysis	Newtonian mechanics, Maxwell equations, Navier-Stokes equations, thermodynamic equations, and other principles

Figure 1.1

Program analysis addresses a basic question common in every engineering area

Use of program analysis is not limited to professional software or its developers. As programming becomes a way of living in a highly

connected digitized environment, citizen programmers can benefit from program analysis, too, to sanity-check their daily program snippets.

The target of program analysis is not limited to executable software, either. Once the object's source language has semantics, program analysis can circumscribe its semantics to provide useful information. For example, program analysis of high-level system configuration scripts can provide information about any existing conflicting requests.

Though the benefits of program analysis are obvious, building a cost-effective program analysis is not trivial, since computer programs are complex and often very large. For example, the number of lines of smartphone applications frequently reaches over half a million, not to mention larger software such as web browsers or operating systems, whose source sizes are over ten million lines. With semantics, the situation is much worse because a program execution is highly dynamic. Programs usually need to react to inputs from external, uncontrolled environments. The number of inputs, not to mention the number of program states, that can arise in all possible use cases is so huge that software developers are likely to fail to handle some corner cases. The number easily can be greater than the number of atoms in the universe, for example. The space of the inputs keeps exploding as we want our software to do more things. Also, constraints that could keep software simple and small quickly diminish because of the ever-growing capacity of computer hardware.

Given that software is in charge of almost all infrastructures in our personal, social, and global life, the need for cost-effective program analysis technology is greater than ever before. We have already experienced a sequence of appalling accidents whose causes are identified as mistakes in software. Such accidents have occurred in almost all sectors, including space, medical, military, electric power transmission, telecommunication, security, transportation, business, and administration. The long list includes accidents, the large-scale Twitter outage (2016), the fMRI software error (2016) that invalidated fifteen years of brain research, the Heartbleed bug (2014) in the popular OpenSSL cryptographic library that allows attackers to read the memory of any server that uses certain instances of OpenSSL, the stack overflow issues that can explain the Toyota sudden unintended acceleration (2004–2014), the Northeast blackout (2003), the explosion of the Ariane 5 Flight 501 (1996), which took ten years and $7 billion to build, and the Patriot missile defense system in

Dhahran (1991) that failed to intercept an incoming Scud missile, to name just a few of the more prominent software accidents.

Though building error-free software may be far-fetched, at least within reasonable costs for large-scale software, cost-effective ways to reduce as many errors as possible are always in high demand.

Static analysis, which is the focus of this book, is one kind of program analysis. We conclude this chapter by characterizing static analysis in comparison with other program analysis techniques.

1.3 Concepts in Program Analysis

The remainder of this chapter characterizes static program analysis and compares it with other program analysis techniques. We provide keys to understand how each program analysis technique operates and to assess their strengths and weaknesses. This characterization will give basic intuitions of the strengths and limitations of static analysis.

1.3.1 What to Analyze

The first question to answer to characterize program analysis techniques is *what programs* they analyze in order to determine *what properties*.

Target Programs An obvious characterization of the target programs to analyze is the programming languages in which the programs are written, but this is not the only one.

- **Domain-specific analyses:** Certain analyses are aimed at specific families of programs. This specialization is a pragmatic way to achieve a cost-effective program analysis, because each family has a particular set of characteristics (such as program idioms) on which a program analysis can focus. For example, consider the C programming language. Though the language is widely used to write software, including operating systems, embedded controllers, and all sorts of utilities, each family of programs has a special character. Embedded software is often safety-critical (thus needs thorough verification) but rarely uses the most complex features of the C language (such as recursion, dynamic memory allocation, and non-local jumps `setjump/longjump`), which typically makes analyzing such programs easier than analyzing general applications. Device drivers usually rely on low-level operations that are harder to

reason about (e.g., low-level access to sophisticated data structures) but are often of moderate size (a few thousand lines of code).

- **Non-domain-specific analyses:** Some analyses are designed without focus on a particular family of programs of the target language. Such analyses are usually those incorporated inside compilers, interpreters, or general-purpose programming environments. Such analyses collect information (e.g., constants variables, common errors such as buffer overruns) about the input program to help compilers, interpreters, or programmers for an optimized or safe execution of the program. Non-domain-specific analyses risk being less precise and cost-effective than domain-specific ones in order to have an overall acceptable performance for a wide range of programs.

Besides the language and family of programs to consider, the way input programs are handled may also vary and affects how the analysis works. An obvious option is to handle source programs directly just like a compiler would, but some analyses may input different descriptions of programs instead. We can distinguish two classes of techniques:

- **Program-level analyses** are run on the source code of programs (e.g., written in C or in Java) or on executable program binaries and typically involve a front end similar to a compiler's that constructs the syntax trees of programs from the program source or compiled files.
- **Model-level analyses** consider a different input language that aims at modeling the semantics of programs; then the analyses input not a program in a language such as C or Java but a description that *models* the program to analyze. Such models either need to be constructed manually or are computed by a separate tool. In both cases, the construction of the model may hide either difficulties or sources of inaccuracy that need to be taken precisely into account.

Target Properties A second obvious element of characterization of a program analysis is the set of semantic properties it aims at computing. Among the most important families of target properties, we can cite safety properties, liveness properties, and information flow properties.

- A **safety property** essentially states that a program will never exhibit a behavior observable within finite time. Such behaviors

include termination, computing a particular set of values, and reaching some kind of error state (such as integer overflows, buffer overruns, uncaught exceptions, or deadlocks). Hence, a program analysis for some safety properties chases program behaviors that are observable within finite time. Historically this class is called *safety property* because the goal of the analysis is to prove the absence of bad behaviors, and the bad behaviors are mostly those that occur on finite executions.

- A **liveness property** essentially states that a program will never exhibit a behavior observable only after infinite time. Examples of such behaviors include non-termination, live-lock, or starvation.

  Hence, program analysis for liveness properties searches for the existence of program behaviors that are observable after infinite time.
- **Information flow properties** define a large class of program properties stating the absence of dependence between pairs of program behaviors. For instance, in the case of a web service, users should not be able to derive the credential of another user from the information they can access. Unlike safety and liveness properties, information flow properties require reasoning about pairs of executions. More generally, so-called *hyperproperties* define a class of program properties that are characterized over several program executions.

The techniques to reason about these classes of semantic properties are different. As indicated above, safety properties require considering only finite executions, whereas liveness properties require reasoning about infinite executions. As a consequence, the program analysis techniques and algorithms dedicated to each family of semantic properties will differ as well.

1.3.2 Static versus Dynamic

An important characteristic of a program analysis technique is *when* it is performed or, more precisely, whether it operates during or before program execution.

A first solution is to make the analysis at run-time, that is, during the execution of the program. Such an approach is called *dynamic*, as it takes place while the program computes, typically over several executions.

Example 1.1 (User assertions) *User assertions provide a classic case of a dynamic approach to checking whether some conditions are satisfied by all program executions. Once the assertions are inserted, the process is purely dynamic: whenever an assertion is executed, its condition is evaluated, and an error is returned if the result is* **false**.

Note that some programming languages perform run-time checking of specific properties. For instance, in Java, any array access is preceded by a dynamic bound check, which returns an exception if the index is not valid; this mechanism is equivalent to an assertion and is also dynamic.

A second solution is to make the analysis *before* program execution. We call such an approach a *static* analysis, as it is done once and for all and independently from any execution.

Example 1.2 (Strong typing) *Many programming languages require compilers to carry out some type-checking stage, which ensures that certain classes of errors will never occur during the execution of the input program. This is a perfect example of a static analysis since typing takes place independently from any program execution, and the result is known before the program actually runs.*

Static and dynamic techniques are radically different and come with distinct sets of advantages and drawbacks. While dynamic approaches are often easier to design and implement, they also often incur a performance cost at run-time, and they do not force developers to fix issues before program execution. On the other hand, after a static analysis is done once, the program can be run as usual, without any slowdown. Also, some properties cannot be checked dynamically. For example, if the property of interest is termination, dynamically detecting a non-terminating execution would require constructing an infinite program run. Dynamic and static analyses have different aftermaths once they detect a property violation. A dynamic analysis upon detecting a property violation can simply abort the program execution or apply an unobtrusive surgery to the program state and let the execution continue with a risk of having behaviors unspecified in the programs. On the other hand, when a static analysis detects a property violation, developers can still fix the issue before their software is in use.

1.3.3 A Hard Limit: Uncomputability

Given a language of programs to analyze and a property of interest, an ideal program analysis would always compute in a fully automated way the exact result in finite time. For instance, let us consider the certification that a program (e.g., a piece of safety-critical embedded software) will never crash due to a run-time error. Then we would like to use a static program analysis that will always successfully catch any possible run-time error,

that will always say when a program is run-time error free, and that will never require any user input.

Unfortunately, this is, in general, impossible.

The Halting Problem Is Not Computable The canonical example of a semantic property for which no exact and fully automatic program analysis can be found is *termination*. Given a programming language, we cannot have a program analysis that, for any program in that language, correctly decides in finite time whether the program will terminate or not.

Indeed, it is well known that the halting problem is *not computable*. We explain more precisely the meaning of this statement. In the following, we consider a *Turing-complete* language, that is, a language that is as expressive as a Turing machine (common general-purpose programming languages all satisfy this condition), and we denote the set of all the programs in this language by L. Second, given a program `p` in L, we say that an execution *e* terminates if it reaches the end of `p` after finitely many computation steps. Last, we say that a program terminates if and only if its executions terminate.

Theorem 1.1 (Halting problem) *The* halting problem *consists in finding an algorithm* `halt` *such that,*

for every program `p` ∈ L, `halt(p)` = **true** *if and only if* `p` *terminates.*

The halting problem is not computable*: there is no such algorithm* `halt`*, as proved simultaneously by Alonso Church [20] and Alan Turing [106] in 1936.*

This means that termination is beyond the reach of a fully automatic and precise program analysis.

Interesting Semantic Properties Are Not Computable More generally, any *nontrivial semantic properties* are also not computable. By *semantic property* we mean a property that can be completely defined with respect to the set of executions of a program (as opposed to a syntactic property, which can be decided directly based on the program text). We call a semantic property nontrivial when there are programs that satisfy it and programs that do not satisfy it. Obviously only such properties are worth the effort of designing a program analysis.

It is easy to see that a particular nontrivial semantic property is uncomputable; that is, the property cannot have an exact decision procedure (analyzer). Otherwise, the exact decision procedure solves the halting problem. For example, consider a property: this program prints `1` and finishes. Suppose there exists an analyzer that correctly decides the

property for any input program. This analyzer solves the halting problem as follows. Given an input program *P*, the analyzer checks its slightly changed version "*P*; `print 1`." That the analyzer says "yes" means *P* stops, and "no" means *P* does not stop.

Indeed, Rice's theorem settles the case that any nontrivial semantic property is not computable:

Theorem 1.2 (Rice theorem) *Let* L *be a Turing-complete language, and let P be a nontrivial semantic property of programs of* L*. There exists no algorithm such that,*

for every program $\mathtt{p} \in \mathsf{L}$*, it returns* **true** *if and only if* p *satisfies the semantic property P.*

As a consequence, we should also give up hope of finding an ideal program analysis that can determine fully automatically when a program satisfies any interesting property such as the absence of run-time errors, the absence of information flows, and functional correctness.

Toward Computability However, this does not mean that no useful program analysis can be designed. It means only that the analyses we are going to consider will all need to suffer some kind of limitation, by giving up on automation, by targeting only a restricted class of programs (i.e., by giving up the *for every program* in theorem 1.2), or by not always being able to provide an exact answer (i.e., by giving up the *if and only if* in theorem 1.2). We discuss these possible compromises in the next sections.

1.3.4 Automation and Scalability

The first way around the limitation expressed in Rice's theorem is to give up on *automation* and to let program analyses require some amount of user input. In this case, the user is asked to provide some information to the analysis, such as global or local invariants (an *invariant* is a logical property that can be proved to be inductive for a given program). This means that the analysis is partly manual since users need to compute part of the results themselves.

Obviously, having to supply such information can often become quite cumbersome when programs are large or complex, which is the main drawback of manual methods.

Worse still, this process may be error prone, such that human error may ultimately lead to wrong results. To avoid such mistakes, program analysis tools may independently verify the user-supplied information. Then, when the user-supplied information is wrong, the analysis tool will simply reject it and produce an error message. When the analysis tool can complete the

verification and check the validity of the user-supplied information, the correctness of the final result will be guaranteed.

Even when a program analysis is automatic, it may not always produce a result within a reasonable time. Indeed, depending on the complexity of the algorithms, a program analysis tool may not be able to scale to large programs due to time costs or other resource constraints (such as memory usage). Thus, *scalability* is another important characteristic of a program analysis tool.

1.3.5 Approximation: Soundness and Completeness

Instead of giving up on automation, we can relax the conditions about program analysis by letting it sometimes return inaccurate results.

It is important to note that *inaccurate* does not mean wrong. Indeed, if the kind of inaccuracy is known, the user may still draw (possibly partly) conclusive results from the analysis output. For example, suppose we are interested in program termination. Given an input program to verify, the program analysis may answer "yes" or "no" only when it is fully sure about the answer. When the analysis is not sure, it will just return an undetermined result: "don't know." Such an analysis would be still useful if the cases where it answers "don't know" are not too frequent.

In the following paragraphs, we introduce two dual forms of inaccuracies (or, equivalently, approximations) that program analysis may make. To fix the notations, we assume a semantic property of interest P and an analysis tool `analysis`, to determine whether this property holds.

Ideally, if `analysis` were perfectly accurate, it would be such that,

for every program $\mathtt{p} \in \mathsf{L}$, $\mathtt{analysis(p)} = \textbf{true} \Leftrightarrow \mathtt{p}$ satisfies P.

This equivalence property can be decomposed into a pair of implications:

$$\begin{cases} \text{For every program } \mathtt{p} \in \mathbb{L}, & \mathtt{analysis(p)} = \textbf{true} \implies \mathtt{p} \text{ satisfies } \mathscr{P}. \\ \text{For every program } \mathtt{p} \in \mathbb{L}, & \mathtt{analysis(p)} = \textbf{true} \impliedby \mathtt{p} \text{ satisfies } \mathscr{P}. \end{cases}$$

Therefore, we can weaken the equivalence by simply dropping either of these two implications. In both cases, we get a partially accurate too that may return either a conclusive answer or a nonconclusive one ("don't know").

We now discuss in detail both of these implications.

Soundness A *sound* program analysis satisfies the first implication.

Definition 1.2 (Soundness) *The program analyzer* `analysis` *is* sound *with respect to property P whenever, for any program* `p` ∈ L, `analysis(p)` = **true** *implies that* `p` *satisfies property P.*

When a sound analysis (or analyzer) claims that the program has property *P*, it guarantees that the input program indeed satisfies the property. We call such an analysis *sound* as it always errs on the side of caution: it will not claim a program satisfies *P* unless this property can be guaranteed. In other words, a sound analysis will reject all programs that do not satisfy *P*.

Example 1.3 (Strong typing) *A classic example is that of strong typing that is used in program languages such as ML, that is based on the principle that "well-typed programs do not go wrong": indeed, well-typed programs will not present certain classes of errors whereas certain programs that will never crash may still be rejected.*

From a logical point of view, the soundness objective is very easy to meet since the trivial `analysis` defined to always return **false** obviously satisfies definition 1.2. Indeed, this trivial analysis will simply reject any program. This analysis is not useful since it will never produce a conclusive answer. Therefore, in practice, the design of a sound analysis will try to give a conclusive answer as often as possible. This is in practice possible. As an example, the case of an ML program that cannot be typed (i.e., is rejected) although there exists no execution that crashes due to a typing error is rare in practice.

Completeness A *complete* program analysis satisfies the second, opposite implication:

Definition 1.3 (Completeness) *The program analyzer* `analysis` *is* complete *with respect to property P whenever, for every program* `p` ∈ L, *such that* `p` *satisfies P,* `analysis(p)` = **true**.

A complete program analysis will accept every program that satisfies property *P*. We call such an analysis *complete* because it does not miss a program that has the property. In other words, when a complete analysis rejects an input program, the completeness guarantees that the program indeed fails to satisfy *P*.

Example 1.4 (User assertions) *The error search technique based on user assertions is complete in the sense of definition 1.3. User assertions let developers improve the quality of their software thanks to run-time checks inserted as conditions in the source code and that are checked during program executions. This practice can be seen as a very rudimentary form of verification for a limited class of safety properties, where a given condition should never be violated. Faults are reported during program executions, as assertion failures. If an assertion fails, this means that at least one execution will produce a state where the assertion condition is violated.*

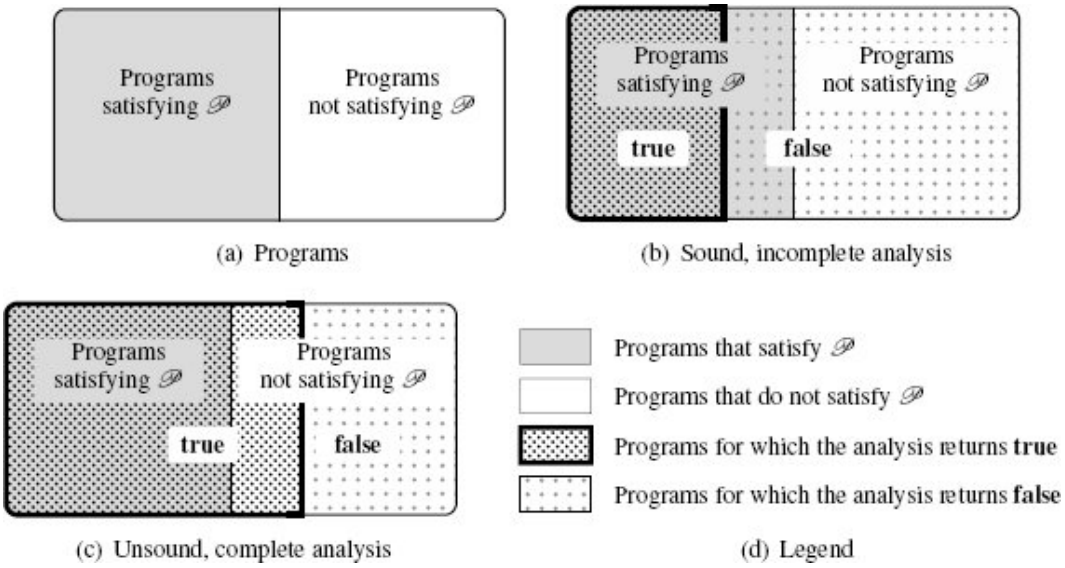


Figure 1.2

Soundness and completeness demonstrated with Venn diagrams

As in the case of soundness, it is very easy to provide a trivial but useless complete analysis. Indeed, if `analysis` always returns **true**, then it never rejects a program that satisfies the property of interest; thus, it is complete, though it is of course of no use. To be useful, a complete analyzer should often reject programs that do not satisfy the property of interest. Building such useful complete analyses is a difficult task in general (just as it is also difficult to build useful sound analyses).

Soundness and Completeness Soundness and completeness are dual properties. To better show them, we represent answers of sound and complete analyses using Venn diagrams in figure 1.2, following the legend in figure 1.2(d):

- Figure 1.2(a) shows the set of all programs and divides it into two subsets: the programs that satisfy the semantic property *P* and the programs that do not satisfy *P*. A sound and complete analysis would always return **true** exactly for the programs that are in the left part of the diagram.
- Figure 1.2(b) depicts the answers of an analysis that is *sound* but *incomplete*: it rejects all programs that do not satisfy the property

but also rejects some that do satisfy it; whenever it returns **true**, we have the guarantee that the analyzed program satisfies *P*.

- Figure 1.2(c) depicts the answers of an analysis that is *complete* but *unsound*: it accepts all programs that do satisfy the property but also accepts some that do not satisfy it; whenever it returns **false**, we have the guarantee that the analyzed program does not satisfy *P*.

Due to the computability barrier, we should not hope for a sound, complete, and fully automatic analysis when trying to determine which programs satisfy any nontrivial execution property for a Turing-complete language. In other words, when a program analysis is automatic, it is either unsound or incomplete. However, this does not mean it is impossible to design a program analysis that returns very accurate (sound and complete) results on a specific set of input programs. But even in that case, there will always exist input programs for which the analysis will return inaccurate (unsound or incomplete) results.

In the previous paragraphs, we have implicitly assumed that the program analysis tool `analysis` always terminates and never crashes. In general, non-termination or crashes of `analysis` should be interpreted conservatively. For instance, if `analysis` is meant to be sound, then its answer should be conservatively considered negative (**false**) whenever it does not return **true** within the allocated time bounds.

1.4 Families of Program Analysis Techniques

In this section, we describe several families of approaches to program analysis. Due to the negative result presented in section 1.3.3, no technique can achieve a fully automatic, sound, and complete computation of a nontrivial property of programs. We show the characteristics of each of these techniques using the definitions of section 1.3.

1.4.1 Testing: Checking a Set of Finite Executions

When trying to understand how a system behaves, often the first idea that comes to mind is to observe the executions of this system. In the case of a program that may not terminate and may have infinitely many executions, it is of course not feasible to fully observe all executions.

Therefore, the *testing* approach observes only a finite set of finite program executions. This technique is used by all programmers, from

beginners to large teams designing complex computer systems. In industry, many levels of testing are performed at all stages of development, such as unit testing (execution of sample runs on a basic function) and integration testing (execution of large series of tests on a completed system, including hardware and software).

Basic testing approaches, such as random testing [11], typically provide a low coverage of the tested code. However, more advanced techniques improve coverage. As an example, *concolic testing* [47] combines testing with symbolic execution (computation of exact relations between input and output variables on a single control flow path) so as to improve coverage and accuracy.

Testing has the following characteristics:

- It is in general easy to automate, and many techniques (such as concolic testing) have been developed to synthesize useful sets of input data to maximize various measures of coverage.
- In almost all cases, it is unsound, except in the cases of programs that have only a finite number of finite executions (though it is usually prohibitively costly in that case).
- It is complete since a failed testing run will produce an execution that is incorrect with respect to the property of interest (such a counter-example is very useful in practice since it shows programmers exactly how the property of interest may be violated and often gives precise information on how to fix the program).

Besides, testing is often costly, and it is hard to achieve a very high path coverage on very large programs. On the other hand, a great advantage of testing is that it can be applied to a program in the conditions in which it is supposed to be run; for instance, testing a program on the target hardware, with the target operating system and drivers, may help diagnose issues that are specific to this combination.

When the semantics of programs is non-deterministic, it may not be feasible to reproduce an execution, which makes the exploitation of the results produced by testing problematic. As an example, the execution of a set of concurrent tasks depends on the scheduling strategy so that two runs with the same input may produce different results, if this strategy is not fully deterministic.

Another consideration is that testing will not allow attacking certain classes of properties. For instance, it will not allow proving that a program terminates, even over a finite set of inputs.

1.4.2 **Assisted Proof: Relying on User-Supplied Invariants**

A second way to avoid the limitation shown in section 1.3.3 consists in giving up on automation.

This is essentially the approach followed by *machine-assisted* techniques. This means that users may be required to supply additional information together with the program to analyze. In most cases, the information that needs to be supplied consists of loop invariants and possibly some other intermediate invariants. This often requires some level of expertise. On the other hand, a large part of the verification can generally still be carried out in a fully automatic way.

We can cite several kinds of program analyses based on machine-assisted techniques. A first approach is based on theorem-proving tools like Coq [24], Isabelle/HOL [49], and PVS [88] and requires the user to formalize the semantics of programs and the properties of interest and to write down proof scripts, which are then checked by the prover. This approach is adapted to the proof of sophisticated program properties. It was applied to the verified CompCert compiler [77] from C to Power-PC assembly (the compiler is verified in the sense that it comes with a proof that it will compile any valid C program properly). It was also used for the design of the microkernel seL4 verified [69]. A second approach leverages a tool infrastructure to prove a specific set of properties over programs in a specific language. The B-method [1] tool set implements such an approach. Also, tools such as the Why C program verification framework [43] or Dafny [76] input a program with a property to verify and attempt to prove the property using automatic decision procedures, while relying on the user for the main program invariants (such as loop invariants) and when the automatic procedures fail.

Machine-assisted techniques have the following characteristics:

- They are not fully automatic and often require the most tedious logical arguments to come from the human user.
- In practice, they are sound with respect to the model of the program semantics used for the proof, and they are also complete up to the abilities of the proof assistant to verify proofs (the expressiveness of the logics of the proof assistant may prevent some programs to be proved, though this is rarely a problem in practice).

In practice, the main limitation of machine-assisted techniques is the significant resources they require, in terms of time and expertise.

1.4.3 Model Checking: Exhaustive Exploration of Finite Systems

Another approach focuses on finite systems, that is, systems whose behaviors can be exhaustively enumerated, so as to determine whether all executions satisfy the property of interest. This approach is called *finite-state model checking* [38, 93, 21] since it will check a model of a program using some kind of exhaustive enumeration. In practice, model-checking tools use efficient data structures to represent program behaviors and avoid enumerating all executions thanks to strategies that reduce the search space.

Note that this solution is very different from the testing approach discussed in section 1.4.1. Indeed, testing samples a finite set of behaviors among a generally infinite set, whereas model checking attempts to check all executions of a finite system.

The finite model-checking approach has been used both in hardware verification and in software verification.

Model checking has the following characteristics:

- It is automatic.
- It is sound and complete *with respect to the model*.

An important caveat is that the verification is performed at the model level and not at the program level. As a first consequence, this means that a model of the program needs to be constructed, either manually or by some automatic means. In practice, most model-checking tools provide a front end for that purpose. A second consequence is that the relation between this model and the input program should be taken into account when assessing the results; indeed, if the model cannot capture exactly the behaviors of the program (which is likely as programs are usually infinite systems since executions may be of arbitrary length), the checking of the synthesized model may be either incomplete or unsound, with respect to the input program. Some model-checking techniques are able to automatically refine the model when they realize that they fail to prove a property due to a spurious counter-example; however, the iterations of the model checking and refinement may continue indefinitely, so some kind of mechanism is required to guarantee termination. In practice, model-checking tools are often conservative and are thus sound and incomplete with respect to the input program. A large number of model-checking tools have been developed for verifying different kinds of logical assertions on various models or programming languages. As an example, UPPAAL [8] verifies temporal logic formulas on timed automata.

1.4.4 Conservative Static Analysis: Automatic, Sound, and Incomplete Approach

Instead of constructing a finite model of programs, *static analysis* relies on other techniques to compute conservative descriptions of program behaviors using finite resources. The core idea is to finitely over-approximate the set of all program behaviors using a specific set of properties, the computation of which can be automated [26, 27]. A (very simple) example is the type inference present in many modern programming languages such as variants of ML. Types [50, 81] provide a coarse view of what a function does but do so in a very effective manner, since the correctness of type systems guarantees that a function of type `int -> bool` will always input an integer and return a Boolean (when it terminates). Another contrived example is the removal of array bound checks by some compilers for optimization purposes, using numerical properties over program variables that are automatically inferred at compile-time. The next chapters generalize this intuition and introduce many other forms of static analyses.

Besides compilers, static analysis has been very heavily used to design program verifiers and program understanding tools for all sorts of programming languages. Among many others, we can cite the ASTRÉE [12] static analyzer for proving the absence of run-time errors in embedded C codes, the Facebook INFER [15] static analyzer for the detection of memory issues in C/C++/Java programs, the JULIA [103] static analyzer for discovering security issues in Java programs, the POLYSPACE [34] static analyzer for ADA/C/C++ programs, and the SPARROW [66] static analyzer for the detection of memory errors in C programs.

Static analysis approaches have the following characteristics:

- They are automatic.
- They produce sound results, as they compute a *conservative* description of program behaviors, using a limited set of logical properties. Thus, they will never claim the analyzed program satisfies the property of interest when it is not true.
- They are generally incomplete because they cannot represent all program properties and rely on algorithms that enforce termination of the analysis even when the input program may have infinite executions. As a consequence, they may fail to prove correct some programs that satisfy the property of interest.

Static analysis tools generally input the source code of programs and do not require modeling the source code using an external tool. Instead, they directly compute properties taken in a fixed set of logical formulas, using algorithms that we present throughout the following chapters.

While a static analysis is incomplete in general, it is often possible to design a sound static analysis that gives the best possible answer on classes of interesting input programs, as discussed in section 1.3.5. However, it is then always possible to craft a correct input program for which the analysis will fail to return a conclusive result.

Last, we remark that it is entirely possible to drop soundness so as to preserve automation and completeness. This leads to a different kind of analysis that produces an under-approximation of the program's actual behaviors and answers a very different kind of question. Indeed, such an approach may guarantee that a given subset of the executions of the program can be observed. For instance, the approach may be useful to establish that this program has at least one successful execution. On the other hand, it does not prove properties such as the absence of run-time errors.

1.4.5 Bug Finding: Error Search, Automatic, Unsound, Incomplete, Based on Heuristics

Some automatic program analysis tools sacrifice not only completeness but also soundness. The main motivation to do so is to simplify the design and implementation of analysis tools and to provide lighter-weight verification algorithms. The techniques used in such tools are often similar to those used in model checking or static analysis, but they relax the soundness objective. For instance, they may construct unsound finite models of programs so as to quickly enumerate a subset of the executions of the analyzed program, such as by considering only what happens in the first iteration of each loop [110], whereas a sound tool would have to consider possibly unbounded iteration numbers. As an example, the commercial tool COVERITY [10] applies such techniques to programs written in a wide range of languages (e.g., Java, C/C++, JavaScript, or Python). Similarly, the tool CODESONAR [79] relies on such approaches so as to search for defects in C/C++ or Assembly programs. The CBMC tool (C Bounded Model Checker) [70] extracts models from C/C++ or Java programs and performs bounded model checking on them, which means

that it explores models only up to a fixed depth. It is thus a case that a model checker gives up on soundness in order to produce fewer alarms.

Since the main motivation of this approach is to discover bugs (and not to prove their absence), it is often referred to as *bug finding*. Such tools are usually applied to improve the quality of noncritical programs at a low cost.

Bug-finding tools have the following characteristics:

- They are automatic.
- They are neither sound nor complete; instead, they aim at discovering bugs rather quickly, so as to help developers.

	Auto-matic	Sound	Complete	Object	When
Testing	Yes	No	Yes	Program	Dynamic
Assisted proving	No	Yes	Yes/No	Model	Static
Model checking of finite-state model	Yes	Yes	Yes	Finite model	Static
Model checking at program level	Yes	Yes	No	Program	Static
Conservative static analysis	Yes	Yes	No	Program	Static
Bug finding	Yes	No	No	Program	Static

Figure 1.3

An overview of program analysis techniques

1.4.6 Summary

Figure 1.3 summarizes the techniques for program analysis introduced in this chapter and compares them based on five criteria. As this comparsion shows, due to the computability barrier, no technique can provide fully automatic, sound, and complete analyses. Testing sacrifices soundness. Assisted proving is not automatic (even if it is often partly automated, the main proof arguments generally need to be human provided). Model-checking approaches can achieve soundness and completeness only with respect to finite models, and they generally give up completeness when considering programs (the incompleteness is often introduced in the

modeling stage). Static analysis gives up completeness (though it may be designed to be precise for large classes of interested programs). Last, bug finding is neither sound nor complete.

As we remarked earlier, another important dimension is *scalability*. In practice, all approaches have limitations regarding scalability, although these limitations vary depending on the intended applications (e.g., input programs, target properties, and algorithms used).

1.5 Roadmap

From now on, we focus on *conservative static analysis*, from its design methodologies to its implementation techniques.

Definition 1.4 (Static analysis) Static analysis *is an automatic technique for program-level analysis that approximates in a conservative manner semantic properties of programs before their execution.*

After a gentle introduction to static analysis in chapter 2, we present a static analysis framework based on a compositional semantics in chapter 3, a static analysis framework based on a transitional semantics in chapter 4, and some advanced techniques in chapter 5. These frameworks, thanks to a semantics-based viewpoint, are general so that they can guide the design of conservative static analyses for any programming language and for any semantic property. In chapter 6, we present issues and techniques regarding the use of static analysis in practice. Chapter 7 discusses and demonstrates the implementation techniques to build a static analysis tool. In chapter 8, we present how we use the general static analysis framework to analyze seemingly complex features of realistic programming languages. Chapter 9 discusses several important families of semantic properties of interest and shows how to cope with them using static analysis. In chapter 10, we present several specialized yet high-level frameworks for specific target languages and semantic properties. Finally, in chapter 11 we summarize this book.

2 A Gentle Introduction to Static Analysis

Goal of This Chapter In this chapter, we provide an introduction to static analysis that does not require any background. This introduction aims at making the core concepts of static analysis intuitive and crisp. To this end, we define a basic programming language that describes sequences of transformations applied to points in a two-dimensional space. [1] The notions presented here extend to realistic programming languages. We formalize them thoroughly in chapter 3 in the case of a basic imperative language.

Recommended Reading: **[S]**, **[D]**, **[U]** We recommend this chapter to all readers, as it introduces the basic intuitions useful to understand many more advanced static analysis techniques and the way static analysis tools work. Understanding these concepts is important not only to design or implement a static analyzer but also to use it as well as possible.

2.1 Semantics and Analysis Goal: A Reachability Problem

Syntax of a Very Basic Language For the sake of making our first introduction to static analysis intuitive, we consider a very basic language with intuitive notions of states and executions. The language under study is inspired by drawing languages used for educational purposes, such as introducing children to programming.

A *state* describes the configuration of a computer running a program, observed at a given instant. In general, this includes a description of the memory contents, the registers, and the program counter. In this chapter, a state will simply denote a point in the two-

dimensional space, described by its real coordinates $(\mathtt{x},\mathtt{y})$. We denote the set of such states by S.

We let programs define combinations of basic geometric operations. Basic operations comprise:

- initialization with a point that is non-deterministically chosen in a fixed region $\mathfrak{R}$ (e.g., the $[0,1] \times [0,1]$ square or any other geometrical shape specified by a set of points);
- geometrical translations (specified by a vector); and
- geometrical rotations (specified by a center and an angle in degrees).

Moreover, a program is defined as a sequence of operations, or as a non-deterministic choice of two sequences of operations, or as a non-deterministic iteration of a sequence of operations (the number of iterations is chosen non-deterministically). To avoid starting from an undefined state, we assume that a program always begins with an initialization, which fixes the set of initial states.

The syntax of programs is defined by the grammar below (note that we only consider programs that start with an initialization statement, even though this grammar does not express that constraint):

p	::= **init**($\mathfrak{R}$)	initialization, with a state in $\mathfrak{R}$
	| **translation**(u,v)	translation by vector (u,v)
	| **rotation**(u,v,θ)	rotation defined by center (u,v) and angle θ
	| p ; p	sequence of operations
	| {p} **or** {p}	choice (the branch taken is non-deterministic)
	| **iter** {p}	iteration (the number of iterations is non-deterministic)

Semantics As observed in chapter 1, static analysis aims at computing semantic properties of programs. Therefore, before we look into the definition of a static analysis, we need to define the *semantics* of programs, which should characterize the program executions. A common way to achieve this is to simply let the semantics be the set of all the program executions. Such a semantics is often called *collecting semantics*.

An *execution* provides a complete view of a single run of the program. Since we assume that a program makes discrete computation steps at every clock tick, it is naturally described by a sequence of states. Each of the basic program constructions in the above grammar is quite simple, so we do not formalize them fully. Intuitively, their semantics is defined as follows:

- The initialization operation simply produces a state in a given region.
- Translation and rotation transformations induce basic execution steps, which perform the corresponding geometric transformations.
- Sequences of operations yield sequences of execution steps.
- Non-deterministic choices and iterations, respectively, select or repeat a block of code and construct executions that can be derived from those of the subprograms.

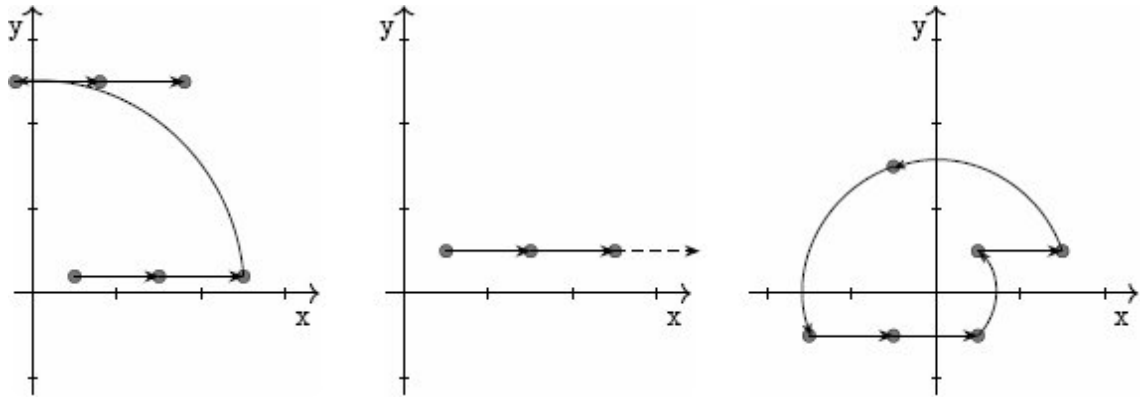


Figure 2.1

A few program executions

Example 2.1 **(Semantics)** *To make this semantics more intuitive, we consider the following program:*

```
init([0,1] × [0,1]);
translation(1,0);
iter{
    {
        translation(1,0)
    }or{
        rotation(0,0,90°)
    }
}
```

This program starts in the $[0,1] \times [0,1]$ *square, performs a translation, and then performs a number of translations or rotations that are chosen non-deterministically (i.e., an oracle decides at run-time both the number of operations and their nature). Figure 2.1 shows three executions:*

- *In the first execution (left), the program starts from* $(0.5,0.2)$*; performs two translations, one rotation, and two translations; and then terminates.*
- *In the second execution (middle), the program starts at point* $(0.5,0.5)$ *and repeats the same transition forever.*
- *In the third execution (right), the program starts at point* $(0.5,0.5)$ *and then repeats forever the sequence made of one translation, two rotations, two translations, and one rotation.*

While the first execution is finite, the other two are actually infinite (which means that the program runs forever).

Semantic Property of Interest: Reachability It is now time to set the property of interest that we are going to consider in this chapter.

We aim for a *reachability* property that is specified by a zone made of points that should not be reached by any execution of the program. Intuitively, we assume that a set of points is fixed and defines a zone that we expect program executions to *never* reach. In other words, if any execution reaches this zone, we would consider it an error. In the following, we search for a static analysis that is able to catch and reject any program with such an erroneous execution. When a program has no such offending execution, the analysis should, as often as possible, accept the program and issue a proof of correctness.

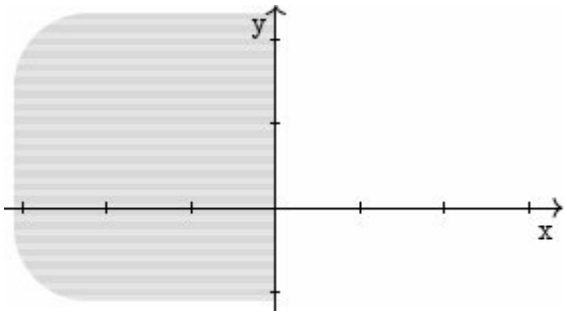


Figure 2.2

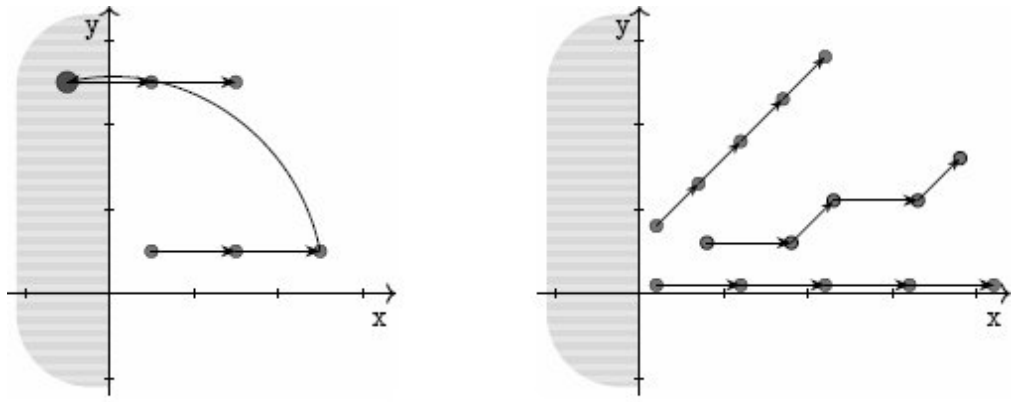
Region supposed to be unreachable: points with a negative x-coordinate

This semantic property is a classic example of safety property (section 1.3.1) (although not all safety properties are of that form). Very often, programmers would like to ensure similar properties in real programs. As an example, reaching a state where a C program will dereference a null pointer will produce an abrupt run-time error. Similarly, if a C program reaches a state where it writes over a dangling pointer, either the execution will fail abruptly or some data will be corrupted. For these reasons, C programmers are usually interested in checking that their programs will never reach a state where they would dereference an invalid, null, or dangling pointer. Thus, the reachability property that we study here is actually quite realistic, even though we are looking at a contrived language.

In this chapter, we often use the region $\mathfrak{D}$ defined by $\mathfrak{D} = \{(\mathrm{x},\mathrm{y}) \mid \mathrm{x} < 0\}$ to denote the set of states that program executions should never reach, although we will construct a program analysis technique that would work for other regions as well. This "error zone" is depicted in figure 2.2. We let $\neg\mathfrak{D}$ denote the property that we would like to verify since it expresses that all the states a program may reach are *not* in $\mathfrak{D}$. To give more intuition, we study a couple of programs.

Example 2.2 (Reachability and incorrect executions) *First, we consider the program of example 2.1. Obviously, it violates the property* $\neg\mathfrak{D}$ *since figure 2.1 shows two executions that eventually reach a point* (x,y)*, where* $\mathrm{x} < 0$*. As an example, figure 2.3(a) displays an execution*

of the program studied in example 2.1, which is incorrect because it reaches the error zone after three steps.



(a) An incorrect execution (b) Correct executions

Figure 2.3

Reachability and programs

Example 2.3 (Reachability and program with only correct executions) *In this example, we study a second program:*

```
init([0,1] × [0,1]);
iter{
    {
        translation(1,0);
    }or{
        translation(0.5,0.5);
    }
}
```

Figure 2.3(b) displays a few executions of this program, and we observe they are all correct, in the sense that they never enter the error zone $\mathfrak{D}$. In fact, we can informally show that all executions of this program will stay in the safe zone $\neg\mathfrak{D}$ at all times:

- *They start at a point (x,y) such that $0 \leq x \leq 1$, which thus satisfies $\neg\mathfrak{D}$.*
- *During a loop iteration, x is increased by either 1 or 0.5 depending on the result of a non-deterministic choice; thus, the coordinate x remains non-negative.*

Static Analysis for Reachability In the rest of this chapter, we define a static analysis (actually, a family of static analyses) that attempts to determine whether an input program satisfies the semantic property $\neg\mathfrak{D}$. The analysis should always return a sound result: if it returns **true** when applied to an input program p, we expect to have the guarantee that no execution of p will ever reach $\mathfrak{D}$. Therefore, a program such as that of example 2.1 will be flagged as "possibly violating the property of interest." Ideally, we would also like the analysis to be precise enough so that it can conclusively report that the program of example 2.3 is correct.

An obvious way to do this would be to enumerate all executions of the input program so as to determine all reachable configurations. But this would not be feasible, as even simple programs (such as those presented in example 2.1 and example 2.3) have infinitely many executions since the set of initial states is infinite, the length of executions is infinite, and the set of possible series of non-deterministic choices is infinite.

Therefore, we will seek other ways to determine the set of all reachable configurations.

2.2 Abstraction

Core Principle of Abstraction In this section, we search for a way to reason about program executions that will produce a superset of the reachable states and that should be rather simple to compute.

To choose this superset, we first try to draw some intuition from the program studied in example 2.3. Notice that in that example, no execution reaches the region $\mathfrak{D}$, and we gave an informal proof of this fact:

- In the beginning, the x-coordinate is non-negative.
- At each step it may only grow, which means that, if it is non-negative, it will remain so.

In the following, we aim at making such reasoning steps automatic. We remark that the steps of this proof do not use all the information present in program states:

- The value of the y-coordinate is completely ignored.

- Only the sign of the x-coordinate is considered in the proof (or, more precisely, the fact that the value of x may be either positive, zero, or negative).

This intuition forms the basis of *abstraction* [26]: by retaining only rather coarse information about program states and considering how the program runs, we can still infer interesting information about the set of all program executions (in this case, that $x \geq 0$ at all times). Such information is captured by a set of logical properties that the analysis may manipulate, using algorithms described in the next section. In example 2.3, the only logical properties that seem to matter in the proof are $x \geq 0$ (used in the case of the program of example 2.3) and the property **true**, which is satisfied by any state (that is needed to describe the states the program of example 2.1 may reach).

Of course, many possible sets of logical properties could be used in such proofs. Thus, we need to make clear what logical properties the analysis may manipulate.

Definition 2.1 (Abstraction) *We call* abstraction *a set A of logical properties of program states, which are called* abstract properties *or* abstract elements. *A set of abstract properties is called an* abstract domain.

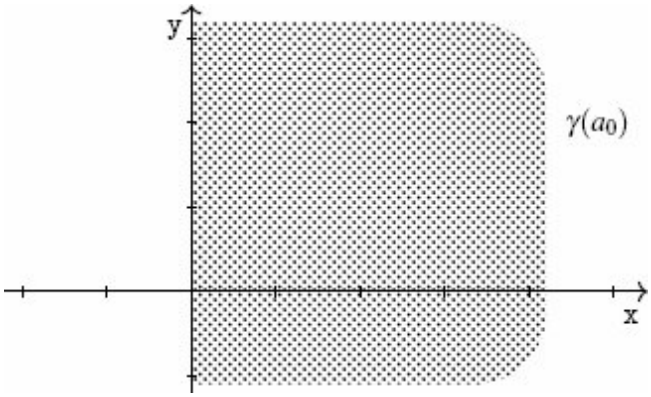


Figure 2.4

Abstraction based on the sign of the x component

In this definition, the word *abstract* is used here as opposed to the word *concrete*; in the following, we use the “concrete” qualifier to

denote actual program behaviors, whereas the “abstract” qualifier applies to the properties used in the (automatic) proofs. As an example, the *concrete semantics* is the actual semantics of programs as defined in section 2.1. By contrast, an *abstract semantics* shall define a computable over-approximation of the concrete semantics expressed in terms of abstract states. Actually, the goal of static analysis is precisely to compute such a sound abstract semantics.

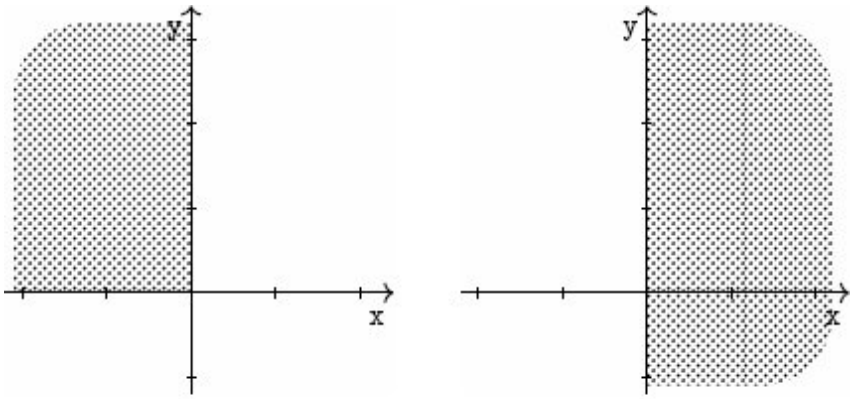
The mathematical and computer representation of abstract elements is crucial for the definition of all the static analysis algorithms that we are going to consider. Ideally, the abstract properties should come with an efficient computer representation and with analysis algorithms (the algorithms are discussed further in this chapter), since we intend to develop a static analyzer that relies on these predicates. Thus, it is important to distinguish the abstract elements from their meaning.

Definition 2.2 (Concretization) *Given an abstract element a of A, we call* concretization *the set of program states that satisfy it. We denote it by $\gamma(a)$.*

Example 2.4 (Abstraction by the sign of the x-coordinate) *As a first example, we present the abstraction used above in order to informally demonstrate that the program of example 2.3 never reaches the region . This abstraction has two elements a_0, a_1, where*

- *a_0 denotes all the states (x,y) such that $x \geq 0$ ($\gamma(a_0)$ is the infinite half-plane area that is filled with dots in figure 2.4).*
- *a_1 denotes the set of all the states such that $\gamma(a_1) = S$ (the whole two-dimensional space).*

In figure 2.4 (and subsequent figures that depict abstract elements), we represent the points described by an abstract element as a zone filled with dots. This region is syntactically different from the abstract element itself: the latter is the representation of the former, and the analysis manipulates only the representation. Even though we often focus less on this distinction in this chapter (and sometimes implicitly assimilate abstract elements and the regions they denote), it will play a great role in subsequent chapters.



(a) Concretization of $[x \leq 0, y \geq 0]$ (b) Concretization of $[x \geq 0]$

Figure 2.5

Signs abstraction

Of course many possible choices of abstractions are less contrived than the one of example 2.4. Some abstractions describe more expressive sets of logical properties than others. Furthermore, some abstractions yield simpler computer representations and less costly algorithms than others. In the following paragraphs, we present a few other examples of abstractions that also have a simple and intuitive graphical representation.

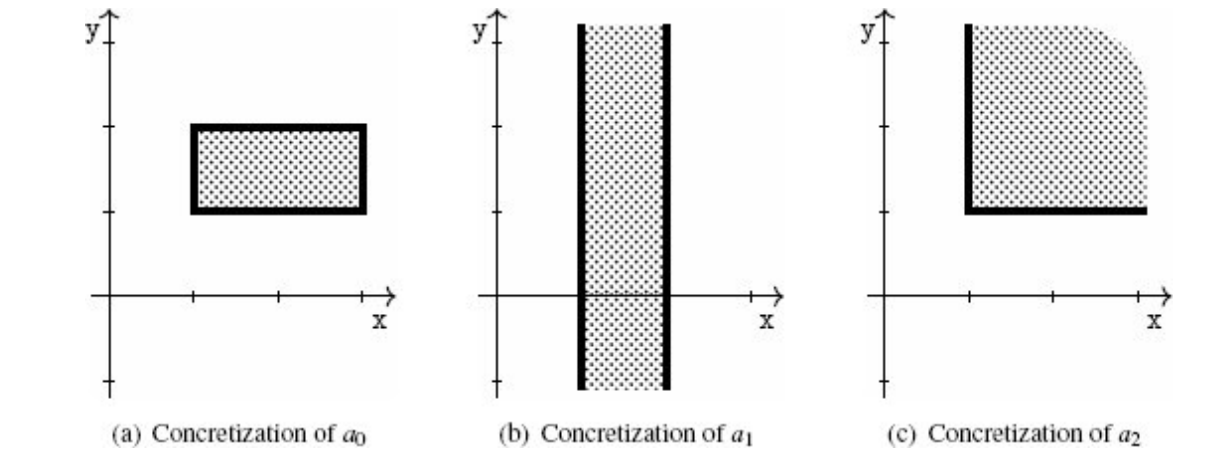
Signs Abstraction The abstraction of example 2.4 treats x and y differently and is very specific to the property $\mathfrak{D}$ defined in section 2.1. It would not work if we wanted a static analysis to prove that y never becomes negative. Similarly, it would not apply if the property to prove was that x does not become positive. However, this abstraction generalizes into a more expressive one, which describes a set of states using two pieces of information: the possible values of the sign of x *and* the possible values of the sign of y. For each variable, this abstraction records whether it may be positive, negative, non-negative, and so forth. The concretizations of a few abstract elements are shown in figure 2.5:

- The left diagram shows the concretization of the abstract element that expresses the fact that x is negative, and y is non-negative.
- The right diagram shows the concretization of the abstract element that expresses the fact that x is positive and this abstract element carries no information about y.

We can observe that this signs abstract domain can express any property the previous domain could express, but it can also describe some properties that were beyond the reach of the previous domain.

Intervals Abstraction In practice, abstractions based on signs are often too weak to capture strong program properties, but other more precise abstractions have been proposed.

Using inequalities and range constraints over variables is a very natural approach to reason over numerical properties. Similarly, we can use range constraints over program variables so as to more precisely describe what values they may take. This is the principle of *intervals abstraction* [26]:



(a) Concretization of a_0 (b) Concretization of a_1 (c) Concretization of a_2

Figure 2.6

Intervals abstraction

Definition 2.3 (Intervals abstraction) *The abstract elements of the interval abstraction are defined by constraints of the form* $l_x \leq x$, $x \leq h_x$, $l_y \leq y$, *and* $y \leq h_y$.

An abstract element is thus composed of at most four finite bound constraints. We remark that such an abstract element may denote the empty set of points, since some sets of constraints cannot be satisfied, such as $1 \leq x$, $x \leq 0$. We do not write down infinite bound constraints (cases where no lower and/or upper bound is given for a given variable).

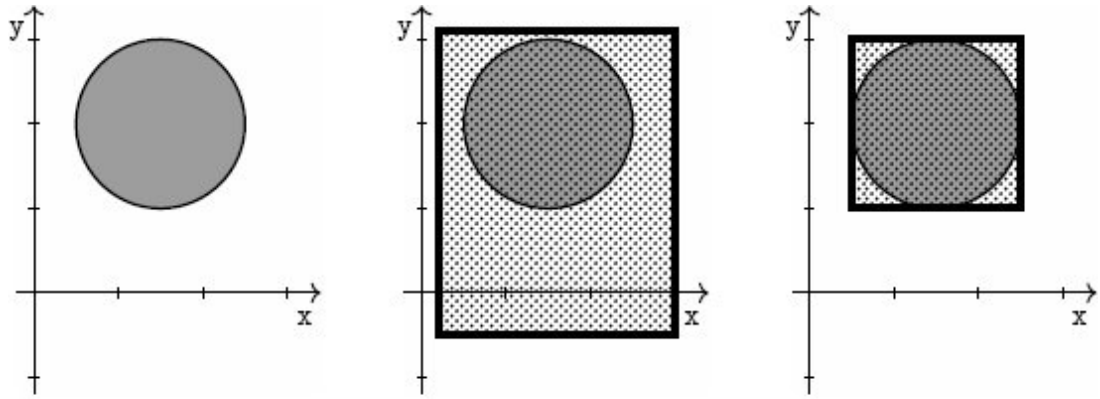
Intuitively, interval abstract domain elements correspond to rectangles in the two-dimensional space, the sides of which are parallel to the axes.

Example 2.5 (Intervals abstraction) *The following three abstract elements illustrate the kind of constraints that can be expressed by the intervals abstract domain, together with their concretizations, shown in figure 2.6:*

- a_0 *corresponds to numerical constraints* $1 \leq x \leq 3$ *and* $1 \leq y \leq 2$ (*the concretization of* a_0 *is shown in figure 2.6(a));*
- a_1 *corresponds to numerical constraints* $1 \leq x \leq 2$ (*the concretization of* a_1 *is shown in figure 2.6(b));*
- a_2 *corresponds to numerical constraints* $1 \leq x$ *and* $1 \leq y$ (*the concretization of* a_2 *is shown in figure 2.6(c)).*

Obviously, the intervals abstract domain is more expressive than the signs abstract domain. Indeed, any abstract element of the signs abstract domain also corresponds to an element in the intervals abstract domain.

The representation of an abstract element of the intervals domain boils down to at most two numerical constants per variable. Thus, this domain over-approximates a set of points in the two-dimensional space with at most four numerical constants, which take very little space in memory during the analysis.



(a) A concrete set (b) Abstractions (c) Best abstraction

Figure 2.7

Best abstraction

We can introduce at this stage the concept of *best abstraction*. Given any set of points (which correspond to program states), we would like to define an abstract element in the intervals abstract domain that over-approximates our initial set. For instance, let us consider the set of program states defined by the disk shown in figure 2.7(a). Then any box that encloses the disk is a valid over-approximation of this set; indeed, any such box describes all the points in the disk (and more), so it provides a conservative approximation of the disk. However, there exist many such enclosing boxes. Yet, some of these abstractions are more desirable than others. As we mentioned earlier, the goal of abstraction is to account for the concrete set of points using a simple description, at the cost of adding some additional points that are not in the concrete set. Adding fewer points that are not in the concrete set is better since it means the abstraction characterizes the set of points in a less ambiguous and more informative way. In the case of the intervals abstract domain, we can actually solve this problem in an elegant manner; indeed, the *smallest* rectangle that encloses any non-empty set of points is well defined, using the greatest lower bounds and least upper bounds over both coordinates (the case of the empty set of

points is trivial, as the empty rectangle is also an element of the abstract domain). In particular, figure 2.7(c) shows the best approximation of the disk.

More generally, the best abstraction [26] is defined as a function that interprets any set of concrete points into an *optimal* abstract element.

Definition 2.4 (Best abstraction) *We say that a is the* best abstraction *of the concrete set S if and only if $S \subseteq \gamma(a)$ and for any a′ that is an abstraction of S (i.e., $S \subseteq \gamma(a')$), then a′ is a coarser abstraction than a. If S has a best abstraction, then the best abstraction is unique. When it is defined, we let α denote the function that maps any concrete set of states into the best abstraction of that set of states.*

As observed above, the intervals abstract domain has a best abstraction function. While computing a precise abstraction (if possible the best abstraction) is preferable in general, we will often encounter useful analyses that cannot compute the best abstraction, or such that the best abstraction cannot even be defined in the sense of definition 2.4. The impossibility to define or compute the best abstraction is in no way a serious flaw for the analysis, as it will only cause it to err on the side of caution (i.e., to return conservative but sound results). Using an over-approximating abstract element is fine, though we prefer to compute the most tightly encompassing one, if it exists.

Finally, we remark that interval constraints cannot capture in a precise manner any complex numerical constraint over both x and y. For instance, it cannot express in an exact manner the property that x is smaller than y. We thus call it a *non-relational abstraction*. Intuitively, an abstract state characterizes each variable by an interval independently from the other variables. On one hand, this simplifies the shape of abstract elements and their representation; on the other hand, it limits the expressiveness of the abstraction.

Convex Polyhedra Abstraction The obvious way to overcome the limitation inherent in the non-relational abstraction is to extend the abstract domain with relational constraints. Augmenting the abstract domain with all linear constraints allows this to be achieved.

Definition 2.5 (Convex polyhedra abstraction) *The abstract elements of the convex polyhedra abstract domain [30] are conjunctions of linear inequality constraints.*

This abstract domain can describe precisely any concrete set that can be described by the signs and intervals abstract domains. It can also describe many other sets of concrete points in a much more precise way than the previous abstractions.

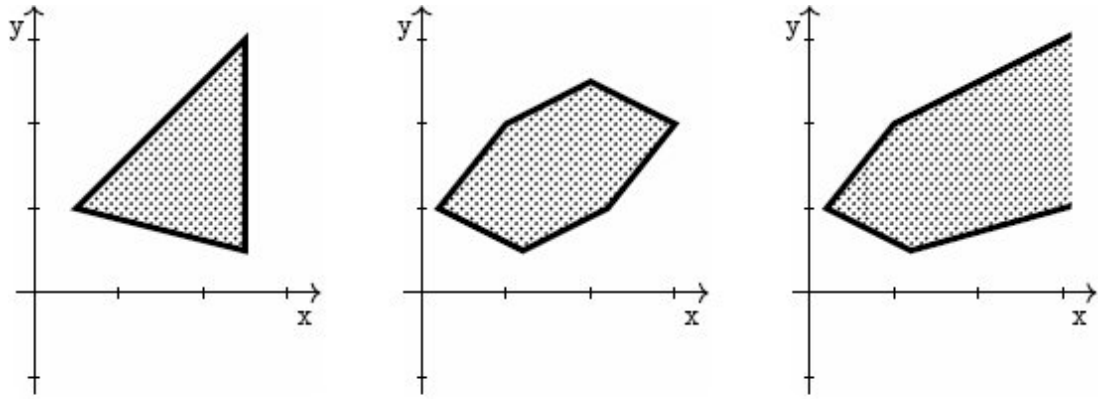
Example 2.6 (Convex polyhedra abstraction) *Figure 2.8 displays the concretization of three convex polyhedra a_0, a_1, and a_2:*

- *a_0 describes the conjunction of the three linear constraints:*

$$\begin{array}{lcll} x & - & y & \geq & -0.5 \\ x & & & \leq & 2.5 \\ x & + & 4y & \geq & 4.5 \end{array}$$

- *a_1 consists of the conjunction of six linear constraints and is of bounded size (we do not list the constraint representation as it would be more involved);*
- *a_2 consists of the conjunction of four linear constraints and describes an unbounded zone (its concretization describes points where x,y may be arbitrarily large).*

There exist several representations for the abstract elements of the convex polyhedra abstract domain. We have already mentioned the representation based on a conjunction of linear inequalities. Since we also noticed that their concretization corresponds exactly to convex polyhedra (hence the name of the abstraction), the abstract elements also have a geometrical representation, based on their sets of vertexes and edges. Actual static analysis algorithms based on convex polyhedra exploit both representations. However, these representations are significantly more complex and costly than for the previous abstract domains: while signs required only a couple of bits per variable, and intervals required at most only two bounds per variable, defining a convex polyhedron typically involves a large number of coefficients or vertexes and edges (in theory there exists no upper bound on the number of constraints).



(a) Concretization of a_0 (b) Concretization of a_1 (c) Concretization of a_2

Figure 2.8

Convex polyhedra abstraction

Another interesting remark about the convex polyhedra abstraction is that concrete sets of points have no best abstraction in general. A disk of diameter 1 provides an example of a concrete set without a smallest enclosing convex polyhedron. On the other hand, *some* concrete sets have a best abstraction (in particular, any set that is a convex polyhedron is its own smallest enclosing convex polyhedron).

In the previous paragraphs, we have provided a few common examples of abstract domains, but many others can be defined and are useful to capture all sorts of constraints (simple or complex, relational or non-relational).

Abstraction of the Semantics of a Program We can now refine the goal of the rest of the chapter. We have set up the notion of abstraction of sets of program states and have shown a few basic abstract domains, adapted to express different kinds of properties. In the next sections, we aim at defining static analysis algorithms to compute in a fully automatic way an over-approximation of the states that a program may reach. Such an over-approximation will be described by an abstract element in one of the abstract domains that we have

sketched. It is called a *conservative abstraction* of the program semantics.

Example 2.7 (Abstractions of reachable states) *We consider the program of example 2.3. Figure 2.9(a) shows all the states that this program may reach; a few program executions were sketched in figure 2.3(b), and we consider here the set of* all *the states that can be reached in at least one execution. We then show the best abstractions that can be computed for this set of states:*

- *Using the intervals abstract domain in figure 2.9(b).*
- *Using the convex polyhedra abstract domain in figure 2.9(c).*

Obviously, the abstraction based on convex polyhedra is much tighter, even though it is still approximate, due to convexity.

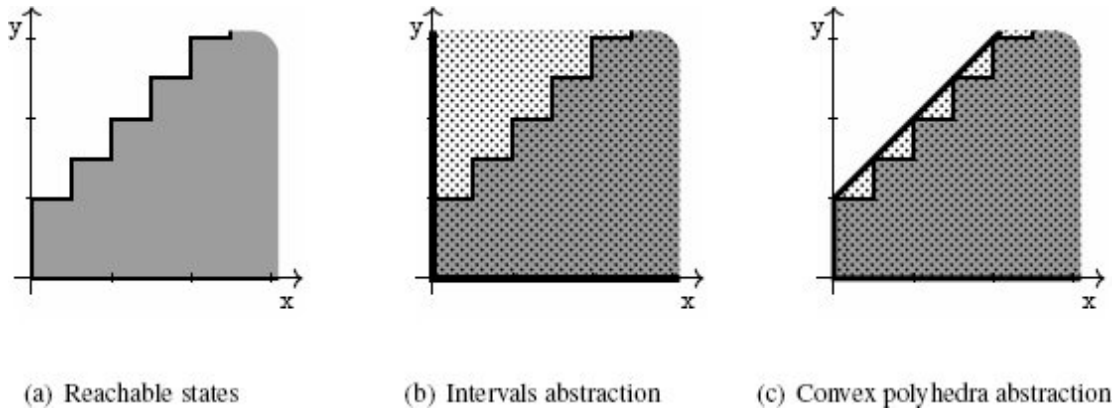


Figure 2.9

Program reachable states and abstraction

As shown in example 2.7, not all abstractions of the semantics of the program are equivalent. Abstractions that describe fewer points are more selective, since they filter out more concrete points, and are thus more likely to help prove that the reachable states are included in a specific set, to prove the property of interest. This means that set inclusion here is fundamental to our study:

- For an abstract element to be a conservative abstraction of the semantics of programs, it should include all the points that are reachable according to the semantics.
- If two abstract elements a_0, a_1 are such that $\gamma(a_0)$ is included into $\gamma(a_1)$, this means that a_0 is more precise than a_1 in the

sense that it allows proving stronger semantic properties.

2.3 A Computable Abstract Semantics: Compositional Style

As the notion of abstraction has been set up in section 2.2, we now show how to derive step-by-step an over-approximation for the states that are visited by a program.

In this section, we introduce a compositional approach to static analysis, based on the step-by-step computation of the effect of each program command. More precisely, given an abstraction of a set of states that denotes a pre-condition (i.e., a set of program execution starting points), we propose to compute an abstract state that over-approximates the set of all the states that may be observed after running that command from the pre-condition (this set is usually called a post-condition). To analyze a sequence of commands, this technique composes the analyses of each sub-command, which is why it is called *compositional*.

Intuitively, this approach incrementally discovers an over-approximation of the set of reachable states of the program. Thus, this also makes it possible to verify that a program never reaches any state in the error zone. Using an accumulator, it is also possible to keep track of an abstraction of all the reachable states of the program.

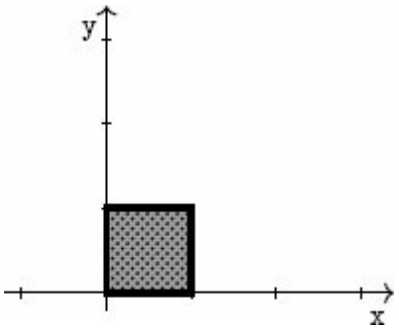


Figure 2.10

Analysis of initialization

2.3.1 Abstraction of Initialization

We start with the effect of the initialization statement that appears at the beginning of programs. At the concrete level, a program initialization statement simply asserts that the initial state of a program execution is located in a given region $\mathfrak{R}$.

To produce an abstraction of the result of initialization, the static analysis simply needs to produce an abstract element that over-approximates the region $\mathfrak{R}$.

When the abstract domain features a best abstraction function α and when the best abstraction of the region $\mathfrak{R}$ is computable, then the abstract element $\alpha(\mathfrak{R})$ provides a solution. This is the case with the intervals abstract domain and with the signs abstract domain.

When the abstract domain does not have a best abstraction function or when this best abstraction is not computable, any abstract element a such that $\gamma(a)$ includes $\mathfrak{R}$ can be chosen. For instance, the convex polyhedra abstract domain does not feature a best abstraction function; however,

- if $\mathfrak{R}$ is a convex polyhedron, then it can be used as an over-approximation of itself;
- otherwise, an enclosing box can be found by using the intervals abstract domain abstraction, and this enclosing box is also an enclosing convex polyhedron, so it also provides an admissible solution (although an imprecise one).

Example 2.8 (Initialization) *We consider the program of example 2.3. Figure 2.10 shows the best abstraction of the initial states, both with the intervals abstract domain and with the convex polyhedra abstract domain. We remark that this abstraction is* exact*; namely, it incurs no loss of precision.*

2.3.2 Abstraction of Post-Conditions

We now discuss basic geometric transformations and try to find a systematic way to over-approximate their output, when given an abstraction of their input. We first fix some terminology:

- An *abstract pre-condition* is an abstraction of the states that can be observed *before* a program fragment.
- An *abstract post-condition* is an abstraction of the states that can be observed *after* that program fragment.

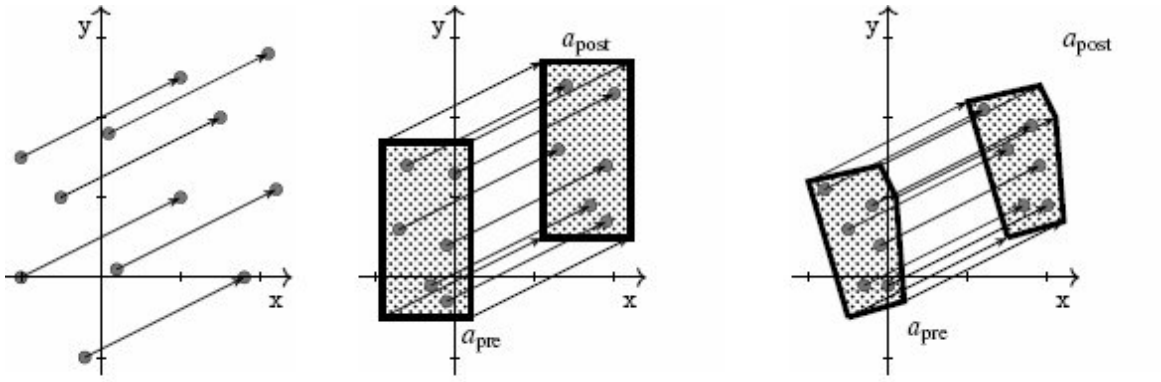


Figure 2.11

Abstraction of the result of a translation

Effect of a Translation We assume an abstract pre-condition a_{pre} and consider program `translation`(u,v). When the program is run in state (x,y) in $\gamma(a_{pre})$, the result is the state $(x + u, y + v)$. Thus, the set of all the images of the points in $\gamma(a_{post})$ can be obtained very simply by translating $\gamma(a_{pre})$ by (u,v). Thus, to produce an over-approximation of the effect of the translation, we simply need to compute an abstract element a_{post} that contains all the points obtained by applying the translation to a point in $\gamma(a_{pre})$.

Example 2.9 (Translation) *We consider* `translation`$(2,1)$ *and the computation of abstract post-condition with a couple of example abstract domains. The effect of the program is shown in figure 2.11(a): any execution boils down to a pair of states.*

Figure 2.11(b) demonstrates the computation of an abstract post-condition with the abstract domain of intervals under the assumption of a given abstract pre-condition. The element a_{post} *is obtained directly from* a_{pre} *by applying translation* $(2,1)$. *If we consider a translation defined by another vector, an abstract post-condition in the intervals abstract domain can be derived from the abstract pre-condition in a similar way.*

The case of the abstract domain of convex polyhedra is similar to the case of intervals, as shown in figure 2.11(c).

In both cases, we note that the abstract post-condition not only contains all the points in the image of the concretization of the pre-condition but also contains no other point*; in this sense, the post-condition is* exact.

Effect of a Rotation We now consider a program of the form **`rotation`**(u,v,θ). The same reasoning toward the design of an automatic algorithm to compute abstract post-conditions from an abstract pre-condition as for the translation still holds. We discuss this transformation in the following example.

Example 2.10 (45° rotation) *To fix the ideas, we assume that* $(u,v) = (0,0)$ *and that* $a = 45°$ *(45° rotation around the origin). A few concrete executions are depicted in figure 2.12(a).*

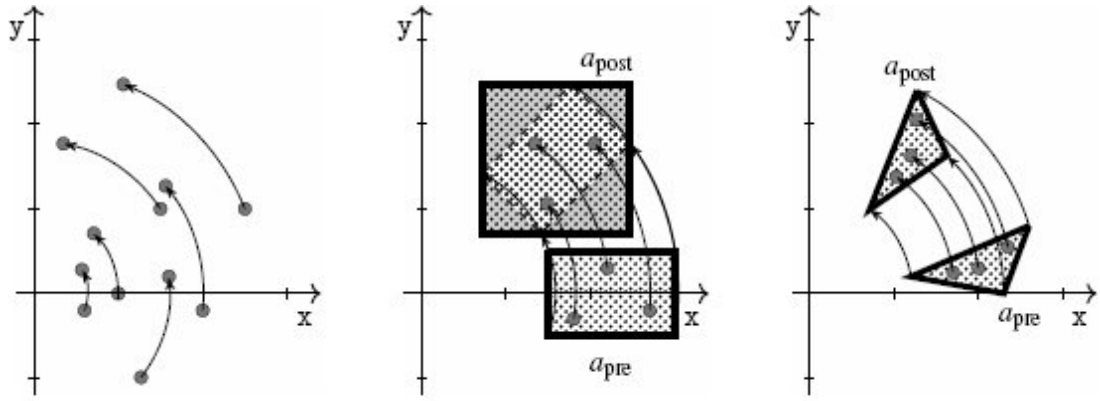


Figure 2.12

Abstraction of the result of a 45° rotation

First, we discuss the case of polyhedra, as it is actually simpler than the case of intervals. Figure 2.12(c) shows that the abstract post-condition can be computed exactly in the same way as for the translation in the previous paragraph. Indeed, if the analysis computes the image of the abstract pre-condition by the rotation, the resulting convex polyhedron contains all the images of the points in the concretization of the pre-condition and thus provides a precise over-approximation of the points that the program may reach after the rotation.

The case of intervals is shown in figure 2.12(b). Intuitively, rotating the pre-condition should give a safe over-approximation of the points that the program may produce after the rotation, but the rotated box is not a valid element of the intervals abstract domain. Indeed, we defined the intervals abstract domain as the set of (finite or infinite) rectangles that are parallel to the axes, and the image of the pre-condition by the rotation is not parallel to the axes. Therefore, to produce a conservative post-condition, that is also an element of the abstract domain, the analysis should produce a bigger box, which is parallel to the axes, as shown in figure 2.12(b). But this result is somewhat imprecise; indeed, as usual, the area filled with dots describes the result of the analysis, and the part of that zone that has a gray background corresponds to points that cannot be observed when running the program from any point in the pre-condition, yet these points have to be included in the result of the analysis

due to the limited expressiveness of the intervals abstraction. Such imprecisions may ultimately prevent the analysis from proving the property of interest.

Conservative Abstract Transfer Functions Based on the two transformations that we have studied so far, we now summarize how the analysis should compute post-conditions in the abstract level. In general, we call an abstract operation that accounts for the effect of a basic program statement a *transfer function*. The definition below formalizes the soundness property that all transfer functions should satisfy.

Definition 2.6 (Sound analysis by abstract interpretation in compositional style) *We consider a static analysis function* `analysis` *that inputs a program and an abstract pre-condition and returns an abstract post-condition. We say that* `analysis` *is sound if and only if the following condition holds:*

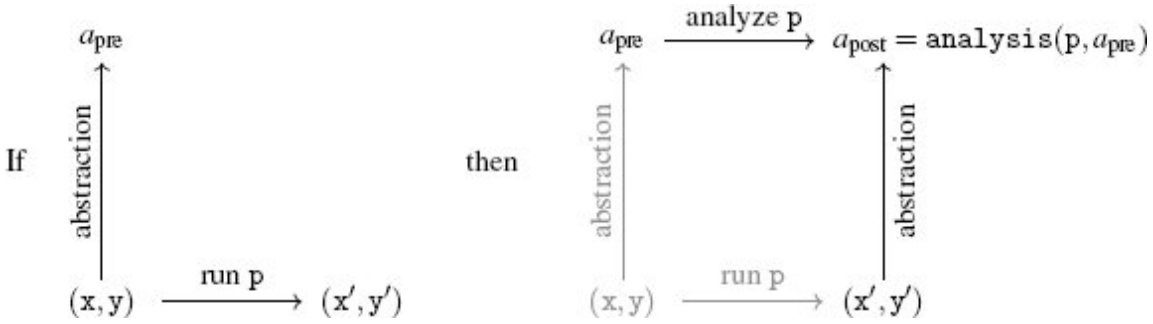


Figure 2.13

Sound analysis of a program p

If an execution of `p` *from a state* $(\texttt{x},\texttt{y})$ *generates the state* $(\texttt{x}',\texttt{y}')$,

then for all abstract element a such that $(\texttt{x}, \texttt{y}) \in \gamma(a)$,

$$(\texttt{x}',\texttt{y}') \in \gamma(\texttt{analysis}(\texttt{p},a))$$

Intuitively, this property states that the analysis should cover all executions of the program: whenever there exists an execution starting from a state that lies inside the abstract pre-condition, the output state should also belong to the abstract post-condition. The diagram of figure 2.13 gives an intuitive presentation of the soundness property: when a concrete state can be described by an abstract pre-condition (bottom to top arrow in the left diagram) and is the starting

point of an execution that reaches a final state (left to right arrow in the left diagram), running the analysis will close the diagram and return an over-approximation of the post-state, as shown in the right diagram.

The transfer functions shown in the previous paragraphs for translations and rotations, for both the interval and polyhedra abstract domains, satisfy the soundness property. Furthermore, we will make sure in the following that the analysis algorithms that we design for other program constructions still preserve this property.

This technique is an instance of *abstract interpretation*: it lets the analysis evaluate each program construction one by one, a bit like a standard interpreter would, albeit in the abstract domain.

At this point, we have defined the following:

$$\begin{aligned} \texttt{analysis}(\textbf{translation}(u,v),a) &= \begin{cases} \text{return an abstract state that contains} \\ \text{the translation of } a \end{cases} \\ \texttt{analysis}(\textbf{rotation}(u,v,\theta),a) &= \begin{cases} \text{return an abstract state that contains} \\ \text{the rotation of } a \end{cases} \end{aligned}$$

Definition 2.6 entails that the analysis will produce sound results in the sense of definition 1.2 when considering the property $\neg\mathfrak{D}$ of interest. Since the analysis over-approximates the states the program may reach, if it claims that $\neg\mathfrak{D}$ is not reachable, then we are sure that the program cannot reach $\neg\mathfrak{D}$.

On the other hand, this definition does not rule out imprecisions. Thus, it accepts analyses that produce coarse over-approximations. In the previous paragraphs we saw both precise analyses and imprecise analyses:

- With the convex polyhedra abstract domain, both the analysis functions for the translation and rotation are precise.
- On the other hand, with the intervals abstract domain, the analysis function for the rotation is imprecise.

Such imprecisions entail that the analysis is not complete in the sense of definition 1.3, and that it may fail to prove that a given region is unreachable.

In the following, we continue the definition of the `analysis` function that can compute sound abstract post-conditions for any program in our language. We proceed by induction over the syntax of programs. Indeed, we have already seen how to handle basic operations (initialization, translations, and rotations); thus, we now consider inductive cases.

The case of sequences of operations is trivial: to compute an abstract post-condition for $\mathtt{p}_0;\mathtt{p}_1$, we start from the abstract pre-condition *a*, compute an abstract post-condition $\mathtt{analysis}(\mathtt{p}_0,a)$ for $\mathtt{p}_0$, and then feed the result as the pre-condition to compute an abstract post-condition for $\mathtt{p}_1$:

$$\mathtt{analysis}(\mathtt{p}_0;\mathtt{p}_1,a) = \mathtt{analysis}(\mathtt{p}_1, \mathtt{analysis}(\mathtt{p}_0, a))$$

The other cases (for non-deterministic choice and iteration) are a bit more complex than the sequence case.

2.3.3 Abstraction of Non-Deterministic Choice

We now assume that $\mathtt{p}_0$ and $\mathtt{p}_1$ are two programs that we already know how to analyze, and we propose constructing a way to over-approximate post-conditions for $\{\mathtt{p}_0\}\mathbf{or}\{\mathtt{p}_1\}$.

Let *a* be an abstract pre-condition, and state $(\mathtt{x},\mathtt{y}) \in \gamma(a)$. Intuitively, we should consider two cases:

- either $\mathtt{p}_0$ is executed, and the result is in $\gamma(\mathtt{analysis}(\mathtt{p}_0,a))$; or
- $\mathtt{p}_1$ is executed, and the result is in $\gamma(\mathtt{analysis}(\mathtt{p}_1,a))$.

Thus, the analysis should simply produce an over-approximation of both $\mathtt{analysis}(\mathtt{p}_0,a)$ and $\mathtt{analysis}(\mathtt{p}_1,a)$.

The computation of an over-approximation for two abstract elements can be done in a systematic way for all the abstract domains that we considered in section 2.2. We can remark that this operation computes an over-approximation for the union of two sets of points viewed as abstract elements. Thus, we denote this abstract operation by `union`. In the case of intervals, the analysis should simply

compute the minimum of lower bounds and the maximum of greatest bounds for both dimensions. In the case of convex polyhedra, it should simply produce a convex hull for both abstract elements. To summarize:

$$\texttt{analysis}(\{\texttt{p}_0\}\,\textbf{or}\,\{\texttt{p}_1\},a) = \texttt{union}(\texttt{analysis}(\texttt{p}_1,a),\texttt{analysis}(\texttt{p}_0,a))$$

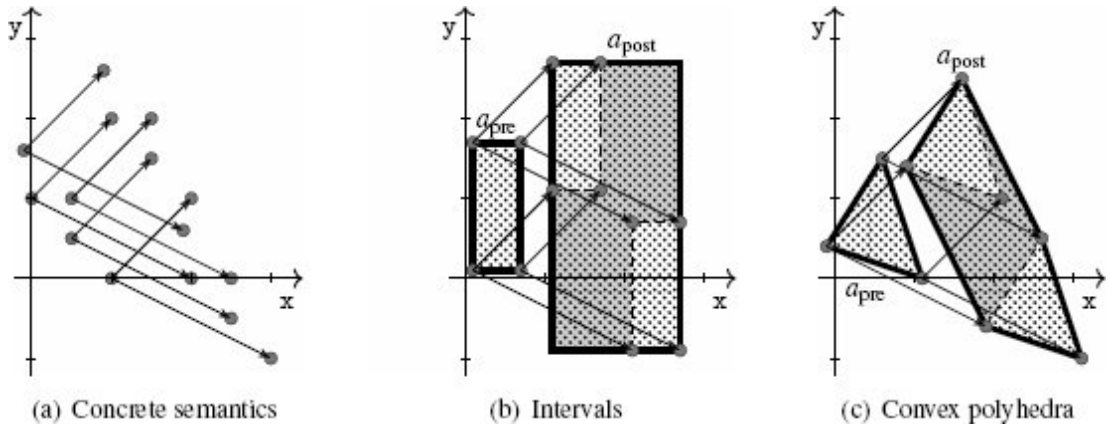


Figure 2.14

Abstraction of the result of a non-deterministic choice

Example 2.11 (Analysis of non-deterministic choice) *In this example, we consider the very simple program below, and we show its analysis with both intervals and convex polyhedra:*

$$\{\textbf{translation}(2,1)\}\,\textbf{or}\,\{\textbf{translation}(-2,-1)\}$$

Figure 2.14(b) shows the computation of an abstract post-condition in the intervals abstract domain, and figure 2.14(c) shows the computation of an abstract post-condition in the convex polyhedra abstract domain. These two cases are quite similar since each branch of the non-deterministic choice boils down to a geometric translation (which induces a translation of the shape of abstract elements), and the analysis should then return an over-approximation of the effects of both branches. In both domains, this operation incurs a significant loss of precision due to the approximation of the convex hull.

The above example shows another reason for the incompleteness of our analysis, as it cannot express precise disjunctive properties. This is a common issue in static analysis, and we present several solutions to this problem in section 5.1.

2.3.4 Abstraction of Non-Deterministic Iteration

Non-deterministic iteration is the last construction that we have to define the analysis for, and is also the most difficult to analyze since it can produce executions of any length and even infinite executions. Therefore, the analysis should compute in *finite time* an over-approximation for *infinitely many arbitrarily long executions*. Still, we observed in example 2.3 that we can derive interesting properties about such programs with rather short informal proofs. Thus, we generalize this approach in this paragraph and design analysis algorithms that compute an over-approximation for the set of output states of a loop.

Note that the abstract post-condition produced as the analysis result describes only the final states of the terminating program executions. This result means that, if a concrete execution terminates, then this abstract post-condition holds. The result does not mean that the iteration will terminate with this abstract post-condition. This is because the halting problem cannot be computed exactly in finite time.

In the following, we consider the following program p that consists of a loop with body b:

$$\mathtt{p} ::= \left\{ \begin{array}{l} \mathtt{iter}\{ \\ \quad \mathtt{b} \\ \} \end{array} \right.$$

We can discriminate the executions of p depending on the number of iterations of the loop; indeed, an execution of program p executes b either zero times, one time, two times, three times, or so on. Thus, p is conceptually equivalent to the following (infinite) program:

```
{}
or{b}
or{b;b}
or{b;b;b}
or{b;b;b;b}
⋮
```

This program fully eliminates the loop and resorts only to the **or** construct, which can be analyzed as described in section 2.3.3, though it obviously cannot be completely written since it would be infinite. However, if we focus on the executions that spend at most k iterations in the loop, we can easily write a program without a loop that has exactly the same behaviors. For any integer k, we let $\mathtt{b}_k$ denote the program that iterates $\mathtt{b}$ k times ($\mathtt{b}_0$ is $\{\}$, $\mathtt{b}_1$ is $\mathtt{b}$, $\mathtt{b}_2$ is $\mathtt{b};\mathtt{b}$, and so on). Moreover, we write $\mathtt{p}_k$ for $\{\mathtt{b}_0\}\mathbf{or}\{\mathtt{b}_1\}\mathbf{or}\ldots\mathbf{or}\{\mathtt{b}_{k-1}\}\mathbf{or}\{\mathtt{b}_k\}$. In short:

```
program p0 is {}
program p1 is {}or{b}
program p2 is {}or{b}or{b;b}
program p3 is {}or{b}or{b;b}or{b;b;b}
                ⋮
```

Then we observe the following equivalence, which relates these programs all together:

$$\mathtt{p}_{k+1} \text{ is equivalent to } \mathtt{p}_k\mathbf{or}\{\mathtt{p}_k;\mathtt{b}\}$$

Indeed, an execution of $\mathtt{p}_{k+1}$ either executes the loop at most k times (hence, it is an execution of $\mathtt{p}_k$), or runs it $k+1$ times exactly (and then it is an execution of $\mathtt{p}_k;\mathtt{b}$). Conversely, one can can prove that an execution of $\mathtt{p}_k\,\mathbf{or}\{\mathtt{p}_k;\mathtt{b}\}$ is also an execution of $\mathtt{p}_{k+1}$.

Therefore, the analysis of this sequence of programs can be computed recursively as follows:

$$\texttt{analysis}(\texttt{p}_{k+1},a) =$$
$$\texttt{union}(\texttt{analysis}(\texttt{p}_k,a),\texttt{analysis}(\texttt{b},\texttt{analysis}(\texttt{p}_k,a)))$$

This approach corresponds to the analysis algorithm that inputs an abstract pre-condition *a*, stores it into a variable R, and iterates the operation:

$$\texttt{R} \leftarrow \texttt{union}(\texttt{R},\texttt{analysis}(\texttt{b},\texttt{R}))$$

Moreover, as shown above, any execution of p can be characterized by the number of times it iterates over the loop. Thus, any execution is ultimately covered by repeating this iterative abstract computation.

The following example illustrates this approach.

Example 2.12 (Abstract iteration) *We consider the program below, which starts at a point located in a triangle and iterates a basic geometric translation a non-deterministically chosen number of times:*

```
init({(x,y) | 0 ≤ y ≤ 2x and x ≤ 0.5});
iter{
    translation(1,0.5)
}
```

We assume that the analysis uses the convex polyhedra abstract domain. Then the set of states observed after initialization and before the **iter** *statement is shown in figure 2.15(b). We show in figure 2.15(c), figure 2.15(d), and figure 2.15(e) the first three iterations of the analysis algorithm sketched above. The imprecision is inherent in the computation of over-approximations (in gray) of abstract elements as in the previous examples.*

While this process does not terminate, we observe that repeating it forever would yield the result shown in figure 2.15(f), which also provides a sound approximation of all the possible output states of the program.

The iterative algorithm demonstrated in example 2.12 will actually always terminate if using the signs abstract domain. Indeed, this abstract domain has a finite number of abstract elements, and when the iterative algorithm computes $\texttt{R} \leftarrow \texttt{union}(\texttt{R},\texttt{analysis}(\texttt{b},\texttt{R}))$, the value of R will converge after finitely many steps: whenever it updates R, either the new value is the same as the previous one (in that case, so will be all the other subsequent values since they are computed using the same formula), or the new value denotes a *strictly* less precise property. Since the number of abstract properties is finite, the latter case will occur at most finitely many times. Therefore, at some

point the value of R stabilizes, and then this value over-approximates the behaviors observed after *any* number of iterations. As a consequence, the termination of signs analysis is guaranteed.

However, we have not solved the issue of termination in the general case yet. The iteration technique used in example 2.12 will obviously not allow for a terminating analysis with the convex polyhedra abstraction or with the interval abstraction.

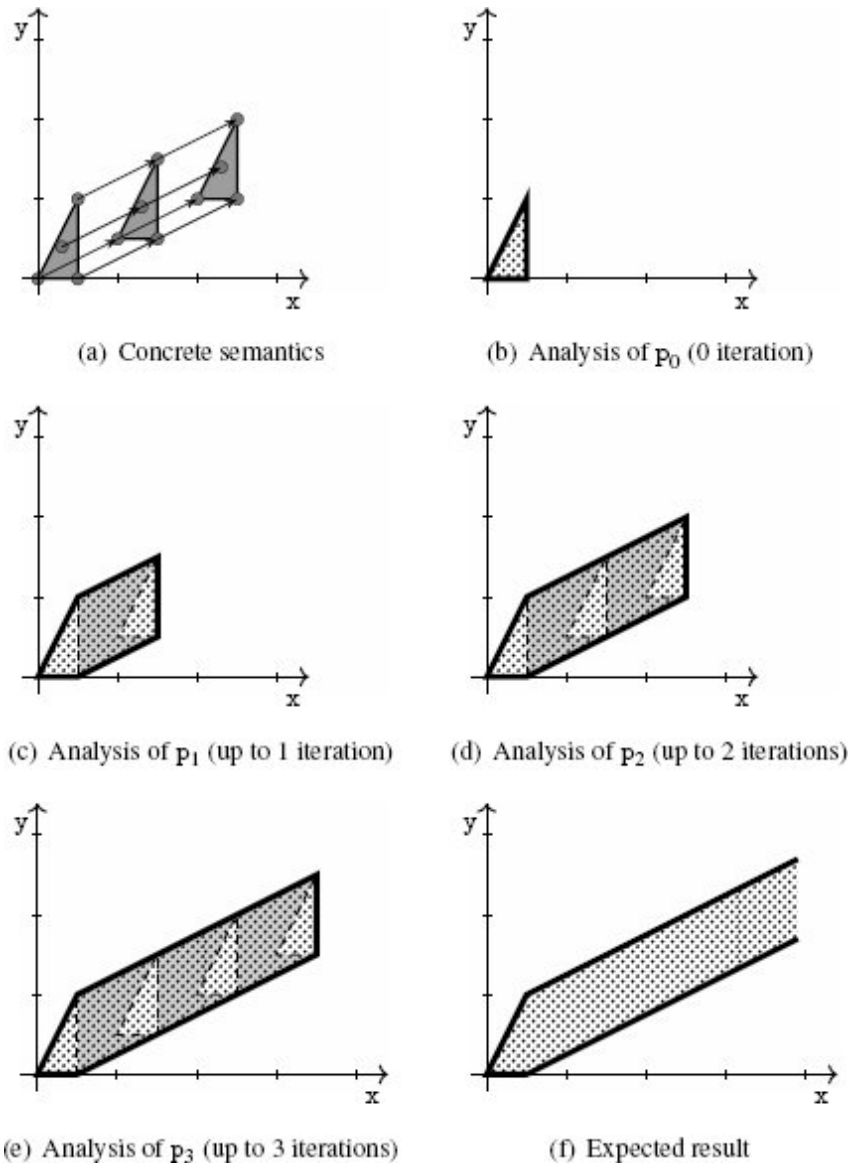


Figure 2.15

Abstract iteration

To ensure termination of the analysis, we need to enforce the convergence of abstract iterates, possibly at the price of a coarser result. Note that a common way to prove that an algorithm terminates involves finding a strictly positive value that decreases strictly over time toward a finitely reachable basis. Thus, a way to enforce the

termination of the analysis is to exhibit such a measure. Very often, non-termination is due to some loop indexes not being incremented properly, preventing such a decreasing measure to exist. Intuitively, this is the issue the iterative analysis algorithm we sketched above suffers from, as shown in example 2.12.

Another interesting observation is that abstract elements are made of finite sets of constraints. Therefore, another way to over-approximate abstract elements that arise in the abstract iteration would consist in forcing this number of constraints to decrease (possibly down to zero) until it stabilizes, thereby recovering termination.

Given the current constraint a_0, suppose that analyzing one more iteration generates a_1. To have an approximate constraint that subsumes both, we can let the analysis:

- keep all constraints of a_0 that are also satisfied in a_1 and
- discard all constraints of a_0 that are not satisfied in a_1 (hence to subsume a_1).

Applying this method to abstract iterates will produce a sequence of abstract elements with a positive, decreasing number of constraints until the sequence stabilizes. This method is an instance of a general technique called *widening*, which enforces the convergence of abstract iterates. We denote this operator by `widen`:

operator `widen` { over-approximates unions; enforces convergence

Stabilization holds when the concretization of the next iterate is included in that of the previous one. For all the abstract domains considered in this chapter, this inclusion can be decided in the abstract level simply by checking geometric inclusion. We thus let `inclusion` denote a function that inputs two abstract elements a_0, a_1 and returns **true** only when it can prove that $\gamma(a_0) \subseteq \gamma(a_1)$:

operator `inclusion` returns **true** only when it succeeds checking inclusion

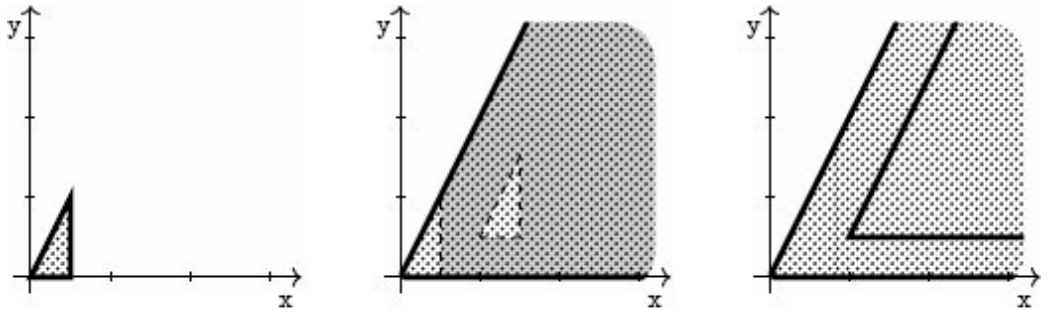
As a conclusion, the following algorithm computes an abstract post-condition for the loop construction:

$$\texttt{analysis}(\textbf{iter}\{\mathrm{p}\}, a) \quad = \quad \begin{cases} \mathrm{R} \leftarrow a; \\ \text{repeat} \\ \qquad \mathrm{T} \leftarrow \mathrm{R}; \\ \qquad \mathrm{R} \leftarrow \texttt{widen}(\mathrm{R}, \texttt{analysis}(\mathrm{p}, \mathrm{R})); \\ \text{until } \texttt{inclusion}(\mathrm{R}, \mathrm{T}) \\ \text{return } \mathrm{T}; \end{cases}$$

This iteration technique will produce a sound result since it over-approximates the abstract elements produced by the sequence of iterates without widening, and its limit (reached after finitely many iterates) also over-approximates all the abstract elements produced by the sequence of iterates without widening and, thus, the states that the program may reach.

The following example illustrates its use in practice.

Example 2.13 (Abstract iteration with widening) *We consider the same program as in example 2.12. Figure 2.15 shows the sequence of abstract iterates using the widening technique. This sequence converges after only two iterations and produces a (rather coarse) over-approximation of the reachable states of the program (shown in figure 2.15(a)). The most interesting point is the computation of the abstract element shown in figure 2.16(b) from the two triangles obtained in the first two iterations:*



(a) Iteration 0 (b) Iteration 1 (c) Iteration 2 and limit

Figure 2.16

Abstract iteration with widening

- *The constraints* $0 \leq y$ *and* $y \leq 2x$ *are stable as they are satisfied in the translated triangle; thus, they are preserved.*
- *The constraint* $x \leq 0.5$ *is not preserved; thus, it is discarded.*

The result obtained in the example clearly shows that widening is another source of imprecision and, thus, of potential incompleteness. Indeed, to ensure convergence in finite time, the analysis weakens the abstract elements more aggressively, adding many points that cannot be observed in any real program execution, as shown in figure 2.16(b).

Fortunately, we can implement many techniques to make the analysis of loops more precise. The example below demonstrates a classic such technique on the same code.

Example 2.14 (Loop unrolling) *We observe that we can rewrite a program with a loop in different ways than the one used so far in this section. In particular, the program of example 2.12 is equivalent to the following program:*

```
init({(x, y) | 0 ≤ y ≤ 2x and x ≤ 0.5});
{}or{
    translation(1, 0.5)
}
iter{
    translation(1, 0.5)
}
```

In essence, analyzing this second version instead has the following effect on the analysis. Indeed, for the first iteration, the `union` *operator will be used, and for all subsequent iterations,* `widen` *will be used instead. When computing widening at iteration 2, all constraints are stable, but the constraint* $x \leq 1.5$*. This produces the result shown in figure 2.17(c). Thus, both the result of the first iteration (shown in figure 2.16(b)) and the widening output (shown in figure 2.17(c)) are much more precise than with the standard widening iteration technique presented in example 2.13.*

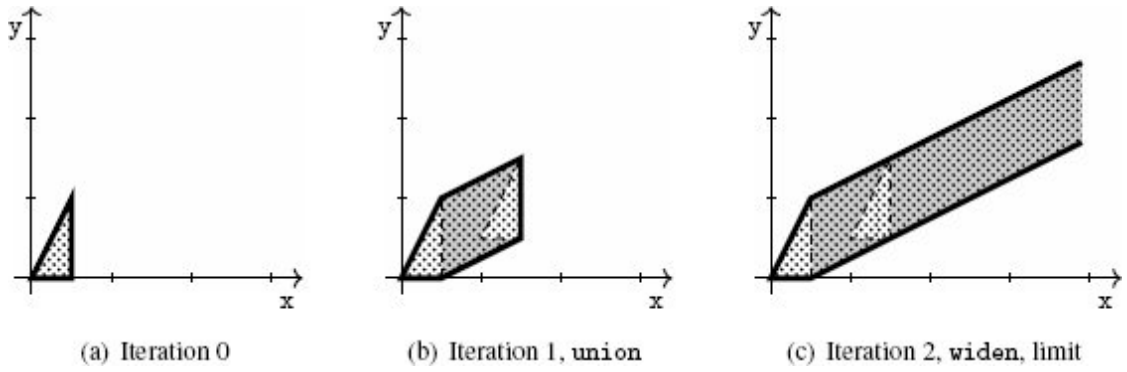


Figure 2.17

Abstract iteration with widening and unrolling

2.3.5 Verification of the Property of Interest

The analysis function that we have designed allows verifying the reachability property of interest introduced in section 2.1.

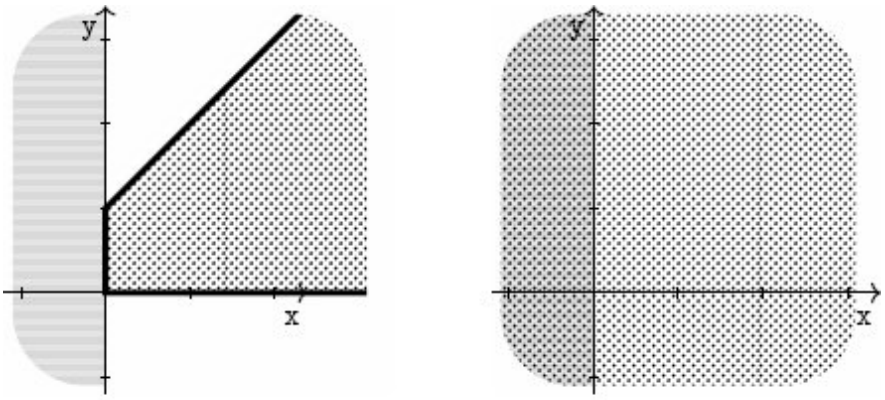
While the analysis function that we have shown so far returns only an over-approximation of the output states (and not of all the intermediate reachable states), it actually computes as intermediate results over-approximations for *all* the reachable states of the input program. Let us consider the case of a sequence $\mathtt{p}_0;\mathtt{p}_1$. The analysis then returns $\mathtt{analysis}(\mathtt{p}_1,\mathtt{analysis}(\mathtt{p}_0,a_{\mathsf{pre}}))$. We observe that, after analyzing $\mathtt{p}_0$ and before analyzing $\mathtt{p}_1$, the analysis holds an over-approximation of all the states that can be observed after executing $\mathtt{p}_0$ and before executing $\mathtt{p}_1$ (the abstract element $\mathtt{analysis}(\mathtt{p}_0,a_{\mathsf{pre}})$). The same holds for each kind of instruction of our language.

As a consequence, the analysis can attempt to verify the property of interest by checking that the abstract elements computed at each step have an empty intersection with $\mathfrak{D}$, or, equivalently, are included in $\neg\mathfrak{D}$. This inclusion can be fully verified in the abstract level, using the same inclusion test we used for checking the termination of the sequences of abstract iterates.

We assume the analysis uses the abstract domain of convex polyhedra and illustrate successful and unsuccessful analyses in the two examples below.

Example 2.15 (Successful verification) *Figure 2.18(a) shows the over-approximation computed for the set of all the reachable states of the program of example 2.3. In this case, the over-approximation* does not *intersect* $\mathfrak{D}$*; thus, the analysis proves the program correct. Again, this result is in line with the conclusion of example 2.7 that this program is correct.*

Example 2.16 (Unsuccessful verification) *Figure 2.18(b) shows the over-approximation computed for the set of all the reachable states of the program of example 2.1 (region filled with dots). Since this zone corresponds to the whole field and intersects the error zone* $\mathfrak{D}$*, the analysis cannot prove the property of interest for this program. This was to be expected. Example 2.2 has shown executions of this program that enter* $\mathfrak{D}$*. While the analysis rightfully flags this program as "potentially wrong," it does not produce a proof that the program is definitely wrong (though we will see in section 5.5 that we can propose static analysis techniques that achieve this proof in certain cases).*



(a) Program of example 2.3 (b) Program of example 2.1

Figure 2.18

Abstractions of reachable states

2.4 A Computable Abstract Semantics: Transitional Style

In section 2.3, the `analysis` function has no explicit machinery to collect all intermediate, reachable states. In other words, it is extensionally defined, analogous to the denotational (or compositional) approach to the semantics. Its inductive definition over

the syntactic structure of the program returns just a post-state of the input program from a pre-state. No collection of intermediate states is manifest in the definition. As discussed in section 2.3.5, however, a simple monitoring mechanism on top of `analysis` can collect all occurring intermediate states.

In this section, we introduce a different style of the analysis function. The new analysis function computes, from the outset, all occurring intermediate states. This formulation is analogous to an operational approach to the semantics.

This transitional style provides us with another convenient perspective that sheds new light on static analysis. In subsequent chapters, we will discuss how the compositional style is better suited for some problems, whereas the transitional style is a better fit for others. Therefore, understanding both styles is beneficial to better grasp static analysis techniques in general.

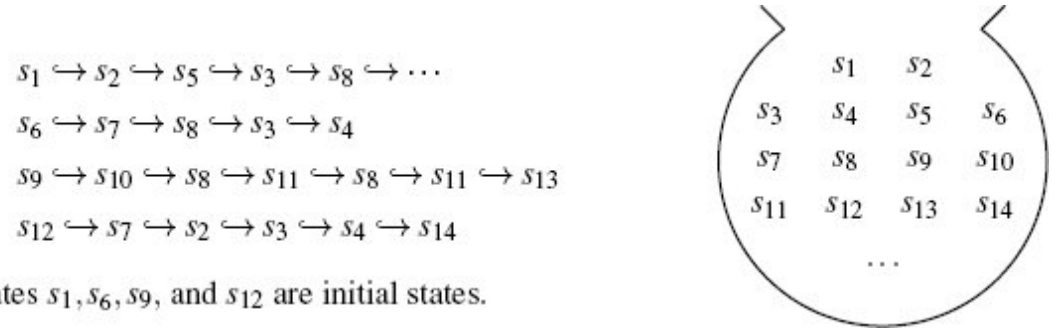


Figure 2.19

Transition sequences and the set of occurring states

2.4.1 Semantics as State Transitions

In the transitional style, we view an execution of a program as a sequence of transitions between states. This transition sequence exposes all the states that occur during the execution.

Let us consider the example language (section 2.1) of this chapter. In this language, a program execution moves a point in the two-

dimensional space. In this case, a state can be defined as a pair comprising a statement label l and a point p in the two-dimensional space.

A single transition

$$(l, p) \hookrightarrow (l', p')$$

between states represents that the program at statement label l transforms the point p to p' and passes it to the next statement label l' for continuation.

One proper transition corresponds to a "single-step" execution of a basic statement. For a compound statement that consists of other statements, its execution consists of the transitions of its sub-statements.

An example of transition sequences for an example program is shown in the next section, after we define how we represent programs and what we mean by "statement labels."

State Transitions and the Collection of all States Let our analysis goal be to collect all the states occurring in all possible transition sequences of the input program. Given such a set of all reachable states, we can check, for example, whether every reachable state remains in a safe zone of our interest.

Figure 2.19 illustrates transition sequences and the collection of states occurring in the sequences. Each node s_i is a state (l, p): a pair comprising a statement label and a point set in the two-dimensional space that is to be transformed by the statement at the label. Here, we schematically show the transition sequences and occurring states. We will show in example 2.17 concrete examples of transition sequences.

Statement Labels and Execution Order We view a program as simply a collection of statements with a well defined execution order. We assign a unique label to each statement of the program. This label can be understood as the so-called *program counter* or *program point*. The execution order, between statements, the so-called *control flow*, is specified by a relation between the labels (from current program points to next program points).

Since our language has a non-deterministic choice and non-deterministic iterations, the execution order is non-deterministic too. Entering the or-statement $\{\mathrm{p}\}\mathbf{or}\{\mathrm{p}'\}$, the next statement to execute is either p or p'. Entering the iteration statement $\mathbf{iter}\{\mathrm{p}\}$, the next statement to execute is either the loop body p or the next statement after the exit of the loop. The next statement of the loop body is again the iteration statement.

For example, consider an example program in figure 2.20. Each statement has a unique label. Figure 2.20(a) shows the program text with statement labels in circles. The statement corresponding to a label is circumscribed by a box with a light contour. Figure 2.20(b) shows a graphical representation of the program with its execution order as directed edges. Rectangular nodes are either basic statements or heads of compound statements. Numbered circle nodes are statement labels.

The non-deterministic function (or relation) for the execution order is defined as follows (as visible in the graph view of figure 2.20(b)):

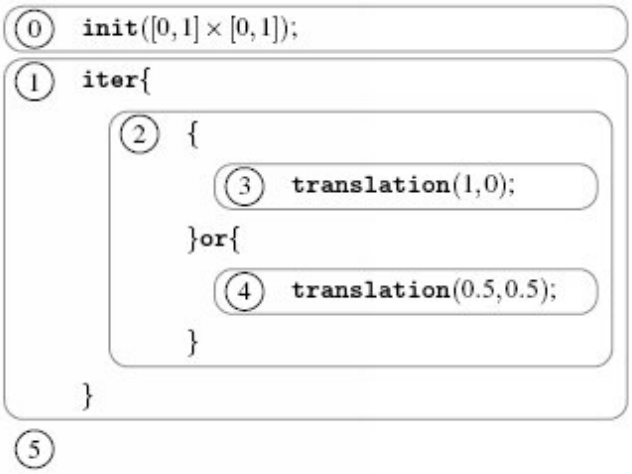
$$\begin{array}{lllllll} \mathtt{next}(0) & = & 1 & & & & \\ \mathtt{next}(1) & = & 2 & \mathtt{next}(1) & = & 5 \\ \mathtt{next}(2) & = & 3 & \mathtt{next}(2) & = & 4 \\ \mathtt{next}(3) & = & 1 & \mathtt{next}(4) & = & 1 \end{array}$$

Note that, in general, for most modern languages the execution order (control flow) is not syntactically obvious. For example, when a language has a dynamic jump construct such as dynamic goto label, dynamic method dispatch, higher-order function call, or the raising of an exception whose target is computed only during execution, the exact execution order is not available before the static analysis. For such languages, determining the execution order should be a part of the static analysis under design or needs to be computed beforehand by another separate static analysis.

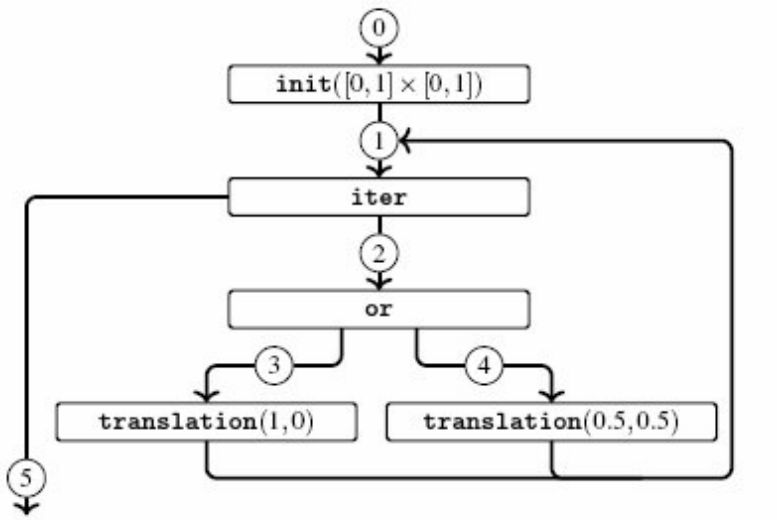
Chapter 4 presents a formal framework that covers such dynamic control-flow cases. Our example language in this chapter is one whose control-flow is obvious from the syntax.

Example 2.17 (Transition sequences) *For the example program in figure 2.20, two examples of state transition ($\hookrightarrow$) sequences starting from statement 0 are as follows: recall that a state*

(*l, p*) *is a pair of a statement label (l) and a point (p) just before being transformed by the corresponding statement. In the following, the left sequence is a transition sequence when the program terminates after two iterations; the right one is when the same program terminates after one iteration:*



(a) Text view, with labels



(b) Graph view, with labels

Figure 2.20

Example program with statement labels

$$
\begin{array}{lcl}
(0, p_0) & \hookrightarrow & (1, p_1) \\
& \hookrightarrow & (2, p_1) \\
& \hookrightarrow & (3, p_1) \\
& \hookrightarrow & (1, p_2) \\
& \hookrightarrow & (2, p_2) \\
& \hookrightarrow & (4, p_2) \\
& \hookrightarrow & (1, p_4) \\
& \hookrightarrow & (5, p_4)
\end{array}
\qquad
\begin{array}{lcl}
(0, p_0) & \hookrightarrow & (1, p_1) \\
& \hookrightarrow & (2, p_1) \\
& \hookrightarrow & (4, p_1) \\
& \hookrightarrow & (1, p_3) \\
& \hookrightarrow & (5, p_3)
\end{array}
$$

where

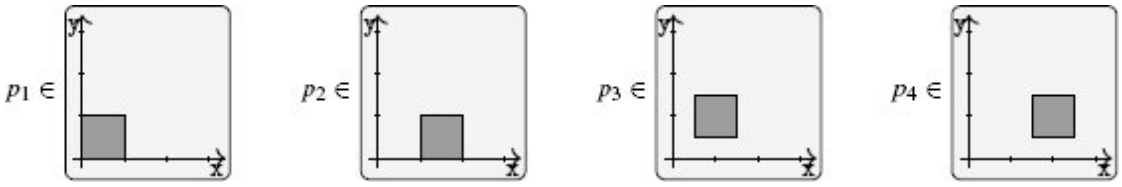


Figure 2.21 shows, on top of the graph view of the program, the right transition sequence.

2.4.2 **Abstraction of States**

Collecting the exact set of all the states that can occur during program executions (transition sequences) is in general either too costly or impossible in finite time. Indeed, due to loops, program executions may be arbitrarily long. Moreover, the set of initial states is also potentially infinite. The situation is worse for other conventional languages that receive inputs from outside. The number of possible inputs is usually combinatorially explosive or even infinite. That is, the number of transition sequences can be infinite too.

Hence, as discussed in section 2.3, the static computation of the set of all possible states cannot be exact in general. Our static computation may be only an approximation in an abstract world.

Now the question is what abstract world we are going to use. As an illustration among many candidates, let us use the following statement-wise abstract world:

> For each statement (program point), an abstract element approximates the set of points that can occur at that program point during executions. The abstract elements for point sets are convex hull pre-conditions as used in section 2.3. In other words, an abstract state is a set of pairs of statement labels and abstract pre-conditions.

Figure 2.22 schematically shows the state abstraction we are using on top of the graphic view of a program. The areas in the two-dimensional plane depict the set of points that can occur during executions.

2.4.3 Abstraction of State Transitions

The abstract state transition is defined over the abstract states of the preceding section. Note that an abstract state is a set of pairs of statement labels and abstract pre-conditions. An abstract transition transforms an abstract state into another abstract state.

Let $Step^{\#}$ be such an abstract state transition function. Given an abstract state X, $Step^{\#}(X)$ returns an abstract post-state. The $Step^{\#}$ function is defined by the one-step abstract transition operator $\hookrightarrow^{\#}$, lifted for a set (for an abstract state):

$$Step^{\#}(X) = \{x' \mid x \in X, x, \hookrightarrow^{\#} x'\}$$

The one-step abstract transition $x \hookrightarrow^{\#} x'$ is the same as the post-condition computations in section 2.3 except that the proper transition happens only for non-compound basic statements, and we reference the `next` function for the next label:

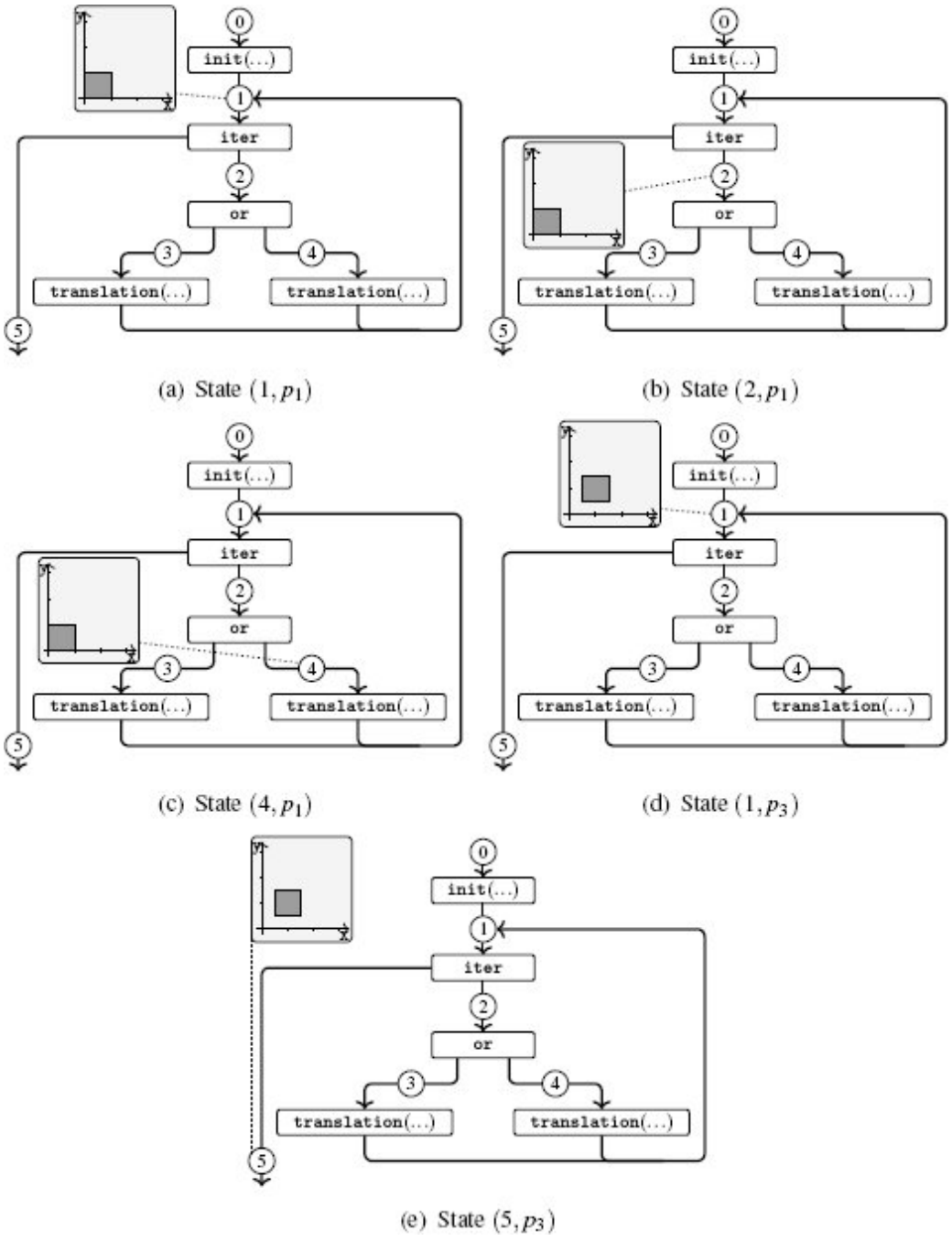


Figure 2.21

States on top of the graph view of the program. Each point p_i belongs to the rectangular area of the corresponding two-dimensional plane.

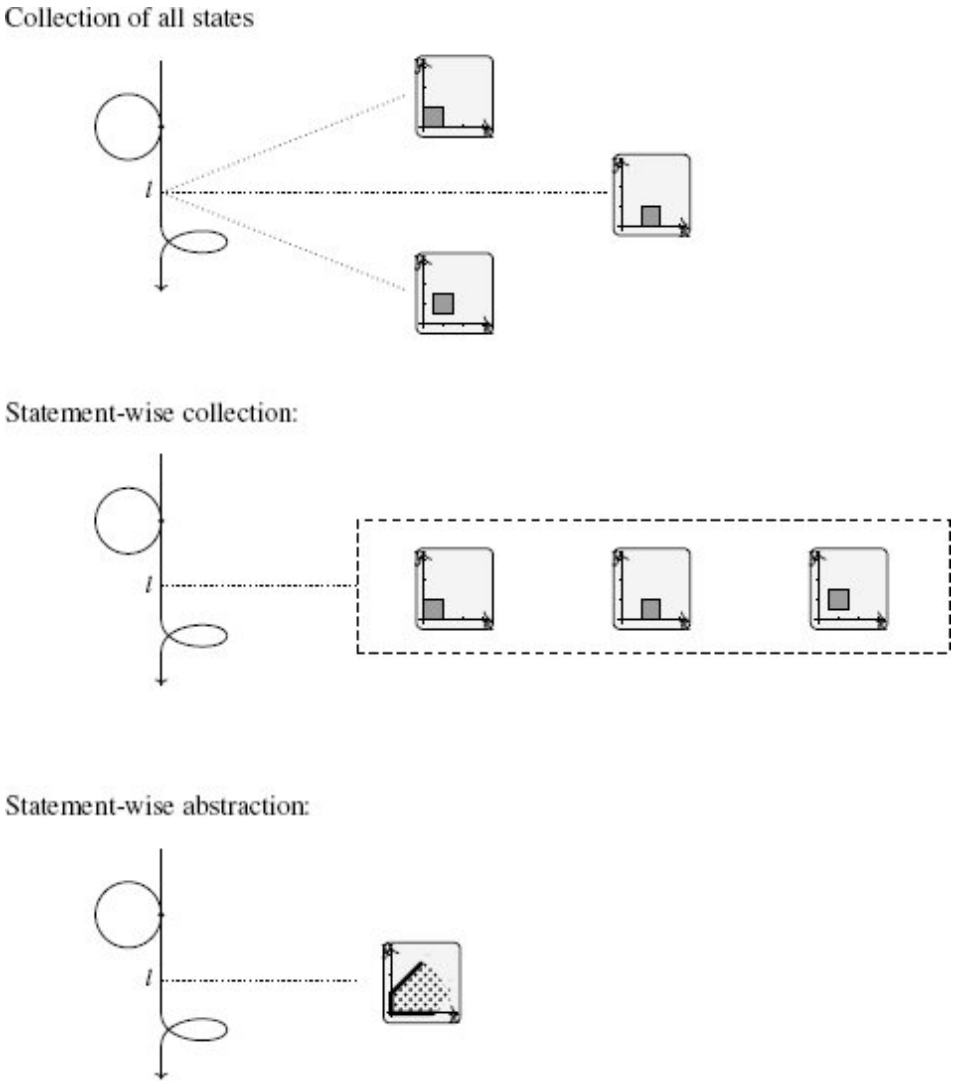


Figure 2.22

Statement-wise abstraction of all the possible states for statement label *l*

$$
\begin{array}{rcll}
(\mathbf{or}_l, a_{\text{pre}}) & \hookrightarrow^\sharp & (\texttt{next}(l), a_{\text{pre}}) & \text{for an } \mathbf{or} \text{ statement at } l \\
 & & & \text{(figure 2.23(a));} \\
(\mathbf{iter}_l, a_{\text{pre}}) & \hookrightarrow^\sharp & (\texttt{next}(l), a_{\text{pre}}) & \text{for an } \mathbf{iter} \text{ statement at } l \\
 & & & \text{(figure 2.23(b));} \\
(\mathrm{p}_l, a_{\text{pre}}) & \hookrightarrow^\sharp & (\texttt{next}(l), \texttt{analysis}(\mathrm{p}_l, a_{\text{pre}})) & \text{else, for a basic statement } \mathrm{p}_l \text{ at } l \\
 & & & \text{(figure 2.23(c)).}
\end{array}
$$

Note that the above *Step*$^\#$ function is sound because the `analysis` function (section 2.3) is sound for basic statements (figure 2.13).

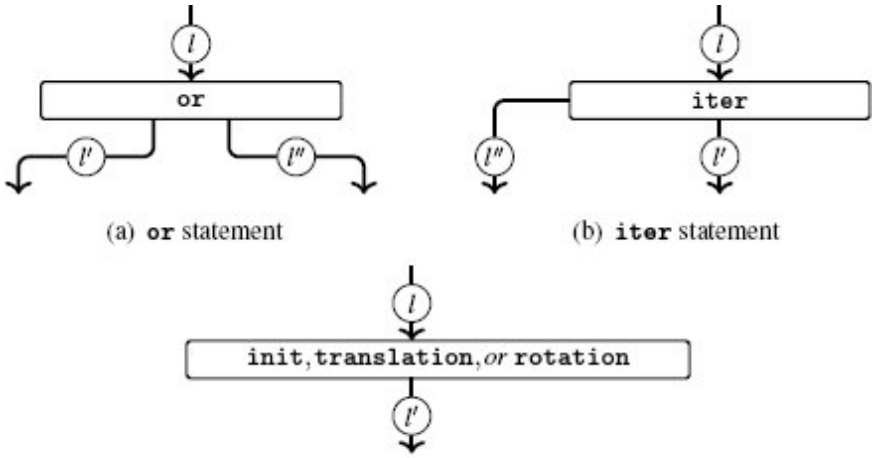


Figure 2.23

Next labels, depending on the statement of *l*, both *l′* and *l″*, or *l′* only

2.4.4 **Analysis by Global Iterations**

The static analysis for collecting all abstract states should be sound. *Soundness* means that the set of concrete states implied by the analysis result over-approximates the reality.

Definition 2.7 **(Sound analysis by abstract interpretation in transitional style)** *Let* `analysis`$_T$ *be a static analysis function in transitional style that inputs a program*

and returns a set of abstract states. We say that $\texttt{analysis}_T$ *is* sound *if and only if the following condition holds:*

> *If S is the set of states occurring in a transition sequence of* $\texttt{p}$ *from initial state* s_0,
>
> *then for any abstract element a such that* $s_0 \in \gamma(a)$,
>
> $$S \subseteq \gamma(\texttt{analysis}_T(\texttt{p}, a)).$$

For the input program $\texttt{p}$ and an abstract state I that over-approximates all the possible initial states, the analysis result $\texttt{analysis}_T(\texttt{p}, I)$ is a set of pairs (l, a_{pre}) of statement label l and abstract pre-condition a_{pre}. The soundness ensures that the abstract pre-conditions at label l over-approximate all the points that may occur at statement l during execution.

Such a sound analysis function $\texttt{analysis}_T(\texttt{p}, I)$ is composed of the sound abstract transition function $Step^\sharp$ in Section 2.4.3 as follows: letting $Step^{\sharp^i}(I)$ denote the abstract post-states after i consecutive abstract transitions from I,

$$\begin{aligned} Step^{\sharp^0}(I) &= I \\ Step^{\sharp^{i+1}}(I) &= Step^\sharp(Step^{\sharp^i}(I)). \end{aligned}$$

This abstract state $Step^{\sharp^i}(I)$ is sound: it subsumes all the states after i transitions from an initial state implied by I. This soundness of i consecutive applications of $Step^\sharp$ is clear because each step by $Step^\sharp$ over-approximates the results of a single-step transition. For our example language, the set I is $\{(0, \textbf{true})\}$, where the label 0 lies at the initial statement and its abstract pre-condition **true** implies all the points in the two-dimensional plane.

Then the analysis accumulates all the abstract states occurring at each step of the abstract transition from the initial abstract state I:

$$Step^{\sharp^0}(I) \cup Step^{\sharp^1}(I) \cup Step^{\sharp^2}(I) \cup \cdots$$

Now, a natural question is how to devise an algorithm that accumulates the above sequence. We remark that the above sequence is equivalent to what we illustrated in section 2.3.4 when we

devised the analysis function for the **`iter`** statement, whose body is now $Step^{\#}$.

The analysis algorithm is to compute the limit ($\lim_{i\to\infty} C_i$) of the following sequence C_i:

$$C_i = Step^{\#0}(I) \cup Step^{\#1}(I) \cup \cdots \cup Step^{\#i}(I).$$

Because the following equivalence holds:

$$C_{k+1} \text{ is equivalent to } C_k \cup Step^{\#}(C_k),$$

the analysis algorithm should simply start from I and iterate the operation

$$\mathtt{C} \leftarrow \mathtt{C} \cup Step^{\#}(\mathtt{C})$$

until stable.

Hence, the analysis algorithm for the input program `p` is a monolithic global iteration:

$$\mathtt{analysis}_T(\mathtt{p}, I) = \begin{cases} \mathtt{C} \leftarrow \{I\} \\ \text{repeat} \\ \quad \mathtt{R} \leftarrow \mathtt{C} \\ \quad \mathtt{C} \leftarrow \mathtt{union}_T(\mathtt{C}, Step^{\sharp}(\mathtt{C})) \\ \text{until } \mathtt{inclusion}_T(\mathtt{C},\mathtt{R}) \\ \text{return R} \end{cases}$$

The i-th iteration of the algorithm covers all states observed up to i execution steps of the input program.

The `union`$_T$ and `inclusion`$_T$ operators do the same as the `union` and `inclusion` operators (section 2.3.4), respectively, except that they are label-wise. That is, the `inclusion`$_T$ (`C`,`R`) returns **true** only when *at every statement label* the local point set implied from `C` is included in that from `R`. Similarly, the `union`$_T$ summarizes *for each statement label* its local set of collected pre-conditions. The

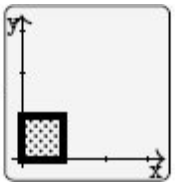
summarization is done by applying the `union` operator to reduce each local set of pre-conditions into a single pre-condition:

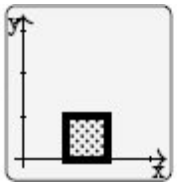
$$\texttt{union}_T(\texttt{C},\texttt{C}') \;=\; \begin{cases} \texttt{X} \leftarrow \{\} \\ \text{for each label } l \text{ in } \texttt{C} \cup \texttt{C}' \\ \quad \texttt{S}_l \leftarrow \{a \mid (l,a) \in \texttt{C}\} \\ \quad \texttt{S}'_l \leftarrow \{a \mid (l,a) \in \texttt{C}'\} \\ \quad \texttt{T}_l \leftarrow \texttt{union}(\texttt{S}_l \cup \texttt{S}'_l) \\ \quad \texttt{X} \leftarrow \texttt{X} \cup \{(l,\texttt{T}_l)\} \\ \text{return X} \end{cases}$$

Example 2.18 (Abstract transitions) *Consider the example program in figure 2.20. Suppose we use the convex polyhedra abstractions for point sets. The above analysis algorithm* `analysis`$_T$ *stores the following* C_i *iterates into the variable* `C` *after i iterations. Remember that* C_i *covers up to i transitions of the input program:*

$$\begin{array}{lll} \textit{after 0 iteration,} & C_0 \overset{let}{=} & \{I\} \\ \textit{after 1 iteration,} & C_1 \overset{let}{=} & \texttt{union}_T(C_0,\{(1,a_1)\}) \\ \textit{after 2 iterations,} & C_2 \overset{let}{=} & \texttt{union}_T(C_1,\{(2,a_1),(5,a_1)\}) \\ \textit{after 3 iterations,} & C_3 \overset{let}{=} & \texttt{union}_T(C_2,\{(3,a_1),(4,a_1)\}) \\ \textit{after 4 iterations,} & C_4 \overset{let}{=} & \texttt{union}_T(C_3,\{(1,a_2),(1,a_3)\}) \\ & \vdots & \vdots \end{array}$$

where

$a_1 =$ 

$a_2 =$ 

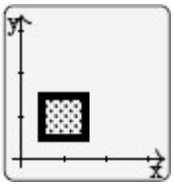
$a_3 =$ 

Figure 2.24 shows the snapshots of computing the iterates C_i *on top of the graph view of the program:*

- *Figure 2.24(a), Figure 2.24(b), and Figure 2.24(c) show the pre-condition* a_1 *at statement labels 1, 2, 3, 4, and 5 until* C_3.
- *Figure 2.24(d), Figure 2.24(e), and Figure 2.24(f) are snapshots of computing* C_4 *from* C_3.
- *Figure 2.24(d) shows two new pre-conditions of statement 1 (post-conditions after statement 3 and 4) resulting from* $\text{Step}^{\#}(C_3)$. *They will be "unioned" with other pre-conditions at statement 1.*

- *Figure 2.24(e) shows the result of unioning a_2 and a_3 during* `union`$\{a_1,a_2,a_3\}$ *at statement 1.*
- *Figure 2.24(f) shows the final result of* `union`$\{a_1,a_2,a_3\}$, `union` *of the above and* a_1.

Analysis Algorithm with the Termination Guarantee We remark that the previous `analysis`$_\mathcal{T}$ algorithm does not guarantee termination. If a program has a loop, the analysis may iterate forever collecting new abstract pre-conditions. To guarantee the termination, we need to use the widening introduced to analyze the iteration statement in section 2.3.4.

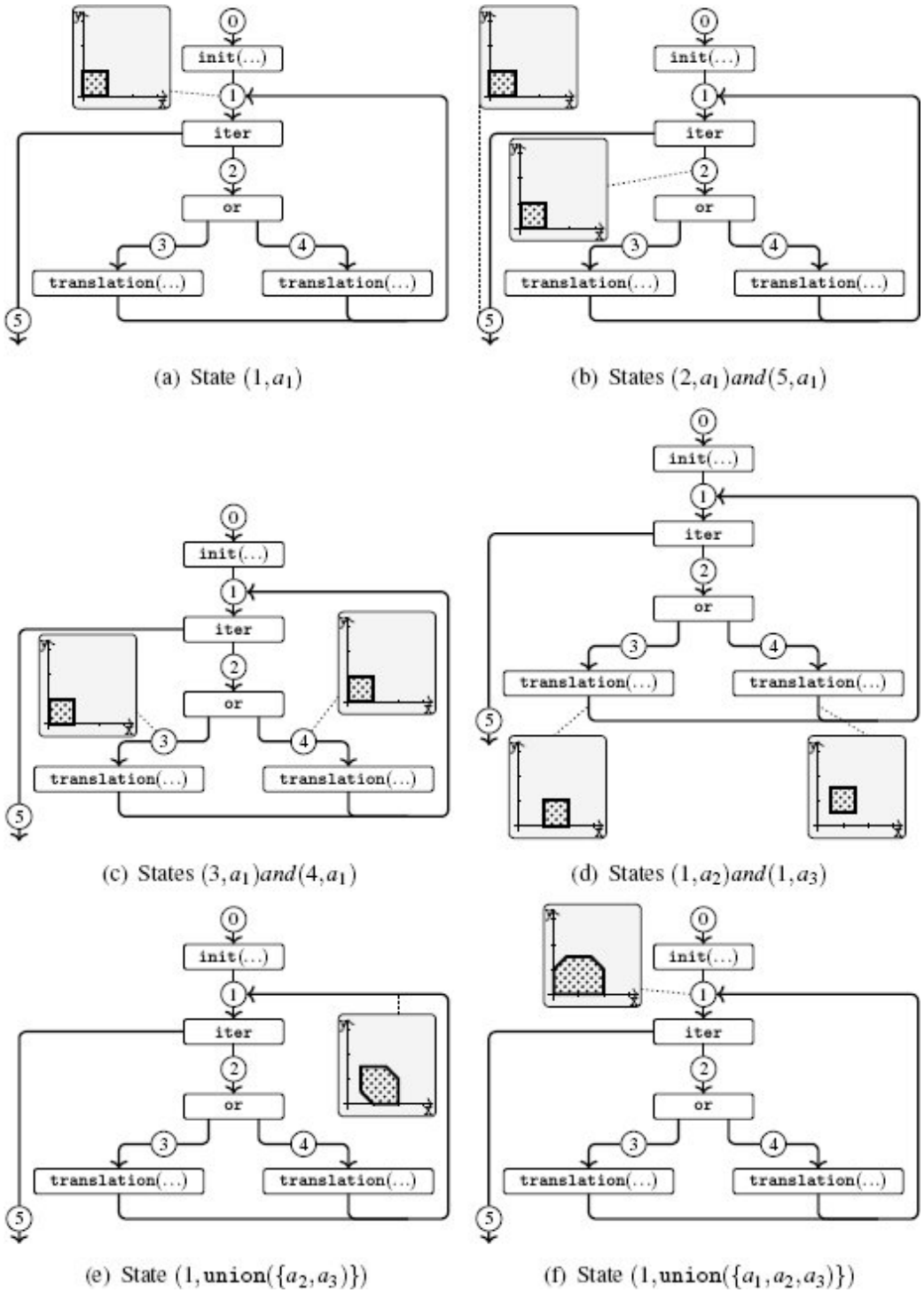


Figure 2.24

Abstract transition snapshots in the graph view of the program

A terminating analysis $\texttt{analysis}_{\mathcal{T}}$ should use a widening operation in place of the $\texttt{union}_{\mathcal{T}}$ operation:

$$\texttt{analysis}_{\mathcal{T}}(\mathrm{p}, I) \;=\; \begin{cases} \mathrm{C} \leftarrow \{I\} \\ \text{repeat} \\ \qquad \mathrm{R} \leftarrow \mathrm{C} \\ \qquad \mathrm{C} \leftarrow \texttt{widen}_{\mathcal{T}}(\mathrm{C}, \mathit{Step}^{\sharp}(\mathrm{C})) \\ \text{until } \texttt{inclusion}_{\mathcal{T}}(\mathrm{C}, \mathrm{R}) \\ \text{return R} \end{cases}$$

The $\texttt{widen}_{\mathcal{T}}$ operator ensures the termination of the sequence of iterations. It makes sure the number of collected constraints for abstract pre-conditions will always decrease.

The $\texttt{widen}_{\mathcal{T}}$ function is identical to the $\texttt{union}_{\mathcal{T}}$ operation except that at the **`iter`** statements we use the `widen` operator in place of the `union` operator. This is because the **`iter`** statement is the only place where iteration happens during program execution. At other statements, we use the `union` operator as before:

$$\texttt{widen}_{\mathcal{T}}(\mathrm{C}, \mathrm{C}') \;=\; \begin{cases} \mathrm{X} \leftarrow \{\} \\ \text{for each label } l \text{ in } \mathrm{C} \cup \mathrm{C}' \\ \qquad \mathrm{S}_l \leftarrow \{a \mid (l, a) \in \mathrm{C}\} \\ \qquad \mathrm{S}'_l \leftarrow \{a \mid (l, a) \in \mathrm{C}'\} \\ \qquad \mathrm{T}_l \leftarrow \text{if the statement at } l \text{ is } \textbf{iter} \\ \qquad\qquad \text{then } \texttt{widen}(\texttt{union}(\mathrm{S}_l), \texttt{union}(\mathrm{S}'_l)) \\ \qquad\qquad \text{else } \texttt{union}(\mathrm{S}_l \cup \mathrm{S}'_l) \\ \qquad \mathrm{X} \leftarrow \mathrm{X} \cup \{(l, \mathrm{T}_l)\} \\ \text{return X} \end{cases}$$

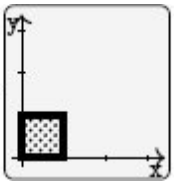
Note that, as opposed to the `union` operator, the `widen` operator is sensitive to the order of its arguments. The `widen`(*a*,*a*′) extrapolates *a* by *a*′ as defined in section 2.3.4. When either argument is **false** (the abstract pre-condition for the empty set of points in the two-

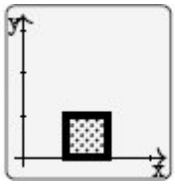
dimensional plane), the widen operation simply returns the other argument.

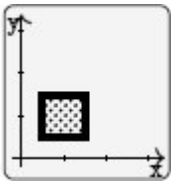
Example 2.19 (Abstract transitions with widening) *Consider again the example program in figure 2.20, and suppose we use the convex polyhedra abstractions for sets of points. The above widening analysis algorithm* `analysis`$_{\mathcal{T}}$ *stores the following iterates* C_i *in the variable* `C` *after i iterations as before but using* `widen`$_{\mathcal{T}}$ *in place of* `union`$_{\mathcal{T}}$*:*

$$\begin{array}{lll}
\textit{after 0 iteration,} & C_0 \stackrel{let}{=} & \{I\} \\
\textit{after 1 iteration,} & C_1 \stackrel{let}{=} & \texttt{widen}_{\mathcal{T}}(C_0, \{(1, a_1)\}) \\
\textit{after 2 iterations,} & C_2 \stackrel{let}{=} & \texttt{widen}_{\mathcal{T}}(C_1, \{(2, a_1), (5, a_1)\}) \\
\textit{after 3 iterations,} & C_3 \stackrel{let}{=} & \texttt{widen}_{\mathcal{T}}(C_2, \{(3, a_1), (4, a_1)\}) \\
\textit{after 4 iterations,} & C_4 \stackrel{let}{=} & \texttt{widen}_{\mathcal{T}}(C_3, \{(1, a_2), (1, a_3)\}) \\
 & \vdots & \vdots
\end{array}$$

where

$a_1 =$ 

$a_2 =$ 

$a_3 =$ 

The widening operation becomes effective at iteration 4 (C_4), when the algorithm brings the effect of the loop body back to the loop head (statement label 1). Figure 2.24(e) shows the abstract state produced when the two results from the **`or`** *statement in the loop body are unioned (*`union`$(\{a_2, a_3\})$*). The algorithm brings this result to the loop head and widens it with the old pre-condition (a_1). In the algorithm, this widening operation*

$$\texttt{widen}(a_1, \texttt{union}(\{a_2, a_3\}))$$

at the loop head happens during the computation of C_4 at iteration 4:

$$\texttt{widen}_{\mathcal{T}}(C_3, \{(1, a_2), (1, a_3)\}).$$

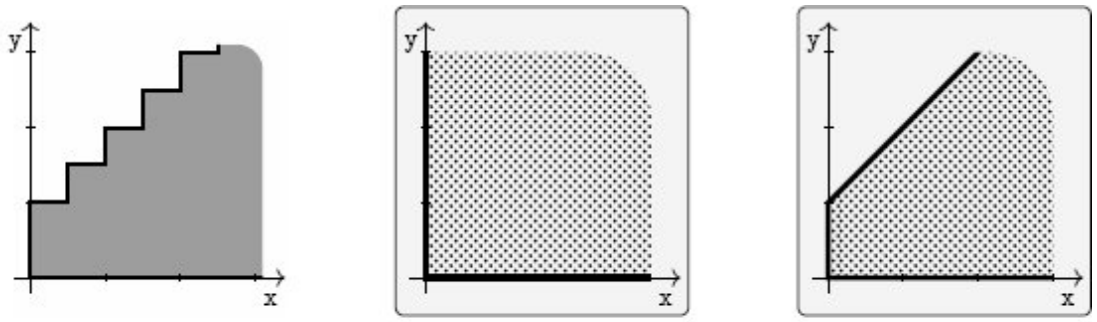
The result corresponds to all the points such that $x \geq 0$ and $y \geq 0$, as shown in figure 2.25(b). It is a rather coarse over-approximation of the actual results, which are shown in figure 2.25(a).

The analysis accuracy can be improved by the "loop-unrolling" technique discussed in example 2.14 (section 2.3.4). This technique rewrites a loop " **`iter`** *{b}" into "{}***`or`***{b};***`iter`** *{b}" before the analysis. The analysis of the unrolled, first iteration ("{}***`or`***{b}") will bring the unioned result to the subsequent loop head. For our example program the analysis result right after the unrolled first iteration is shown in Figure 2.24(f). Widening it at the subsequent loop head with the analysis result of the loop body will generate the result shown in figure*

2.25(c). This result is still an over-approximation of the reality, yet it is more accurate than the result shown in figure 2.25(b).

2.5 Core Principles of a Static Analysis

The previous sections sketch the design of static analyses in the context of a simplistic graphical language. First, in section 2.1 we selected semantic properties of interest and formalized the semantics of programs, with respect to which these properties should be proved. Then in section 2.2 we showed how to define abstractions of the standard semantics of programs. Last, in sections 2.3 and 2.4, we presented two static analyses for this graphical language. In fact, both analyses were derived from a presentation of the semantics of programs (one in compositional style and one in transitional style).



(a) Actual reachable set (b) A first approximation (c) Better approximation

Figure 2.25

Static analysis results

This three-stage approach is actually general and has many fundamental and practical advantages, both for designing and for using static analysis tools.

Indeed, let us first recall the role of each stage:

1. **Selection of the semantics and properties of interest:** This stage is critical as it fixes the goal of the analysis. It describes the behaviors of programs and the properties that

need to be verified. This description is often formal, even though we did it with prose in this chapter.

2. **Choice of the abstraction:** The abstraction describes the properties that are supposed to be manipulated by the analysis. These properties should be strong enough to express the properties of interest and all the invariants required to infer these properties.
3. **Derivation of the analysis algorithms from the semantics and from the abstraction:** The analysis algorithms follow from the choices made in the first two phases for the semantics and for the abstraction. In the two analyses presented in this chapter, we have observed that the analysis closely follows the semantics. For instance, the compositional analysis (section 2.3) follows steps similar to those of a basic program interpreter, in the same order, but using abstract domain predicates instead of regular states.

From the static analysis point of view, this approach puts the choice of the reference semantics and property of interest at the forefront of the design process, as it should be, since this semantics and property define the actual goal of the analysis. It also addresses the selection of the predicates to use before the design of the algorithms to compute these predicates, although it does not preclude revising these choices after testing the analysis, as discussed at the end of this section.

As observed in sections 2.3 and 2.4, most of the choices related to the analysis algorithms are dictated by the abstraction and by the way programs get evaluated according to the concrete semantics. Therefore, this construction also allows justifying the soundness of the analysis step-by-step; indeed, whenever we defined the way a program construction should be handled by the analysis, we ensured that the analysis does not forget any program behavior, according to the abstraction. Thus, the mathematical proof of soundness follows the design of the analysis closely. We discuss this more in chapter 3.

Similarly, this approach also allows tying the analysis and the "standard" semantics of programs. It is actually possible to follow the

same process when implementing a static analyzer, as we show in chapter 7.

Lastly, this methodology also simplifies the troubleshooting of the analysis when it falls short, either in terms of precision (ability to compute strong invariants and achieve the proof of the property of interest) or in terms of scalability (ability to cope with input programs that are large enough). In particular, let us consider the case where the analysis fails to prove the property of interest. Then after investigating the analysis results, the user can diagnose which step "went wrong":

- The first point to check is that the base semantics allows expressing all the steps needed to prove the property of interest and that the abstraction preserves them; indeed, if the abstraction throws important information away, there is no hope that the analysis algorithms will infer predicates that the abstraction cannot capture and will recover from the loss of precision.
- When the semantics and abstraction are strong enough, the imprecisions stem from the analysis algorithms, and one needs to identify which abstract operation (for instance, the computation of the abstract post-condition for some basic operations in the language or the computation of an over-approximation for union) discards important information, which causes the analysis to fail.

When the analysis tool can be parameterized, the user can often remedy such issues by choosing settings carefully. We discuss this point in more depth in chapter 6.

---

[1] This chapter has been partially inspired by graphical descriptions of the notion of abstraction, such as the one presented in http://web.mit.edu/16.399/www/lecture_13-abstraction1/Cousot_MIT_2005_Course_13_4-1.pdf, even though this chapter focuses on different aspects of static analysis, transfer functions, and abstract iterations.